

Техническое задание — документ согласований

Этапы работы

Этап 1 — Завершён

Анализ общего описания: версионность, пространства, права, персонажи.

Этап 2 — Завершён

Анализ типов правил: группировка, расширяемость, пересечения.

Этап 3 — Завершён

Анализ размерных чисел: реализация, хранение, граничные случаи.

Этап 4 — Завершён

Анализ создания персонажа: процесс, статусы, валидация.

Этап 5 — Завершён

Скрытые проблемы: БД, конкурентность, UX, масштабирование.

Этап 6 — Завершён

Авторизация и управление пользователями.

Этап 7 — Завершён

Интерфейс — структура разделов, редакторы, лейаут.

Этап 8 — Завершён

Схема базы данных, индексы, недостающие таблицы.

Этап 1: Согласованные решения

Ключевые сущности

- **Пользователь** — учётная запись (имя, логин, пароль). Набор полей может расширяться.
- **Группа пользователей** — набор прав. Пользователь может быть в нескольких группах, права суммируются.
- **Правило** — единица контента. Имеет **глобальный ID** (сквозной для пространств и времён).
- **Пространство** — изолированная среда правил. По умолчанию «Разработка» и «Актуальные правила». Редактирование возможно в любом пространстве.
- **Версия правила** — комбинация `(rule_global_id, space_id, created_at)`. Две версии одного правила в разных пространствах — разные. Версии не перетираются.
- **Версия правил** (набор) — состояние всех правил в данном пространстве на данный момент времени.
- **Персонаж** — привязан к версии правил (`space_id`, `created_at`). Сам версионизируется.
- **Игра** — обёртка над версией правил. К ней крепятся персонажи той же версии.

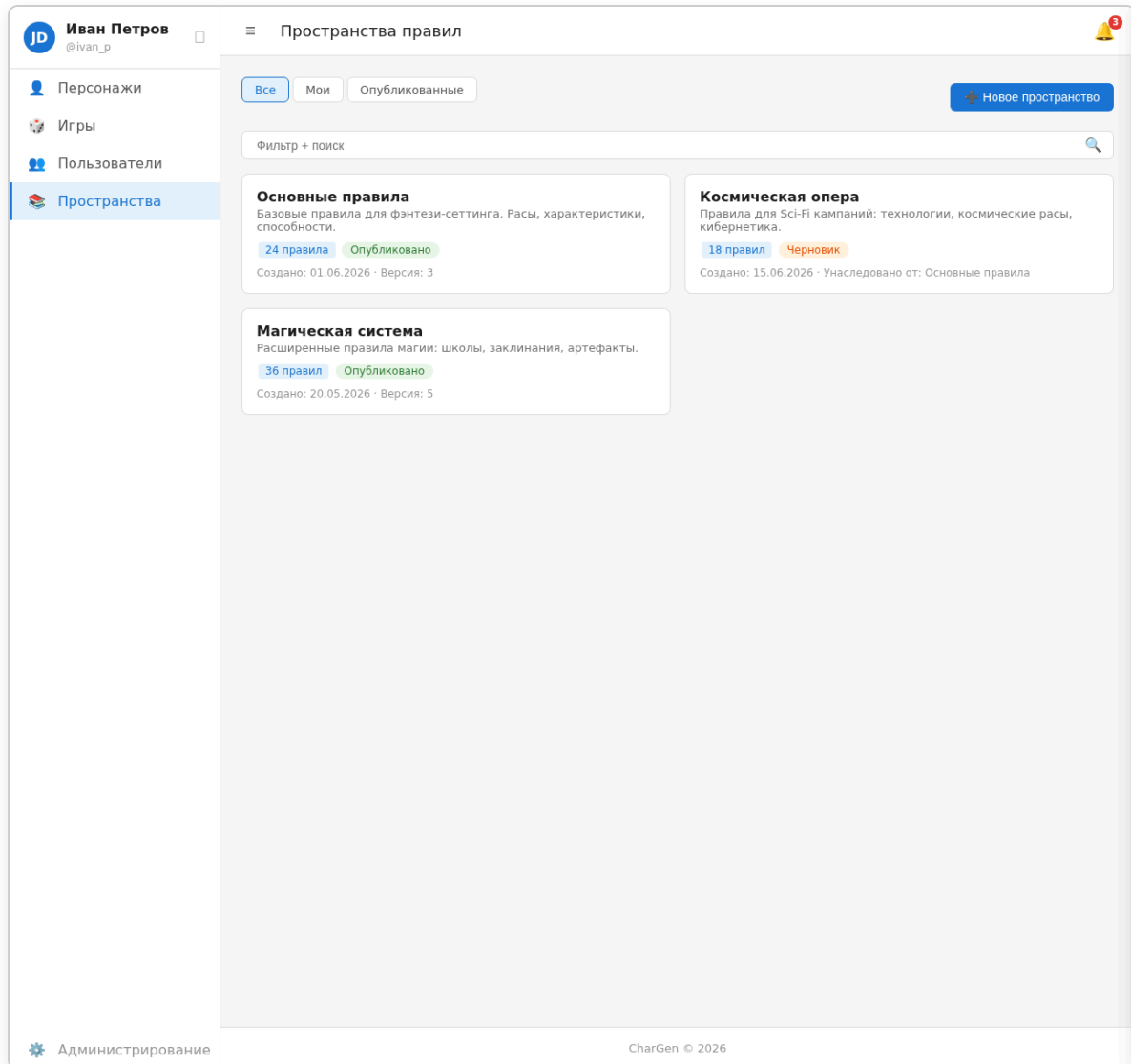
Версионность правил

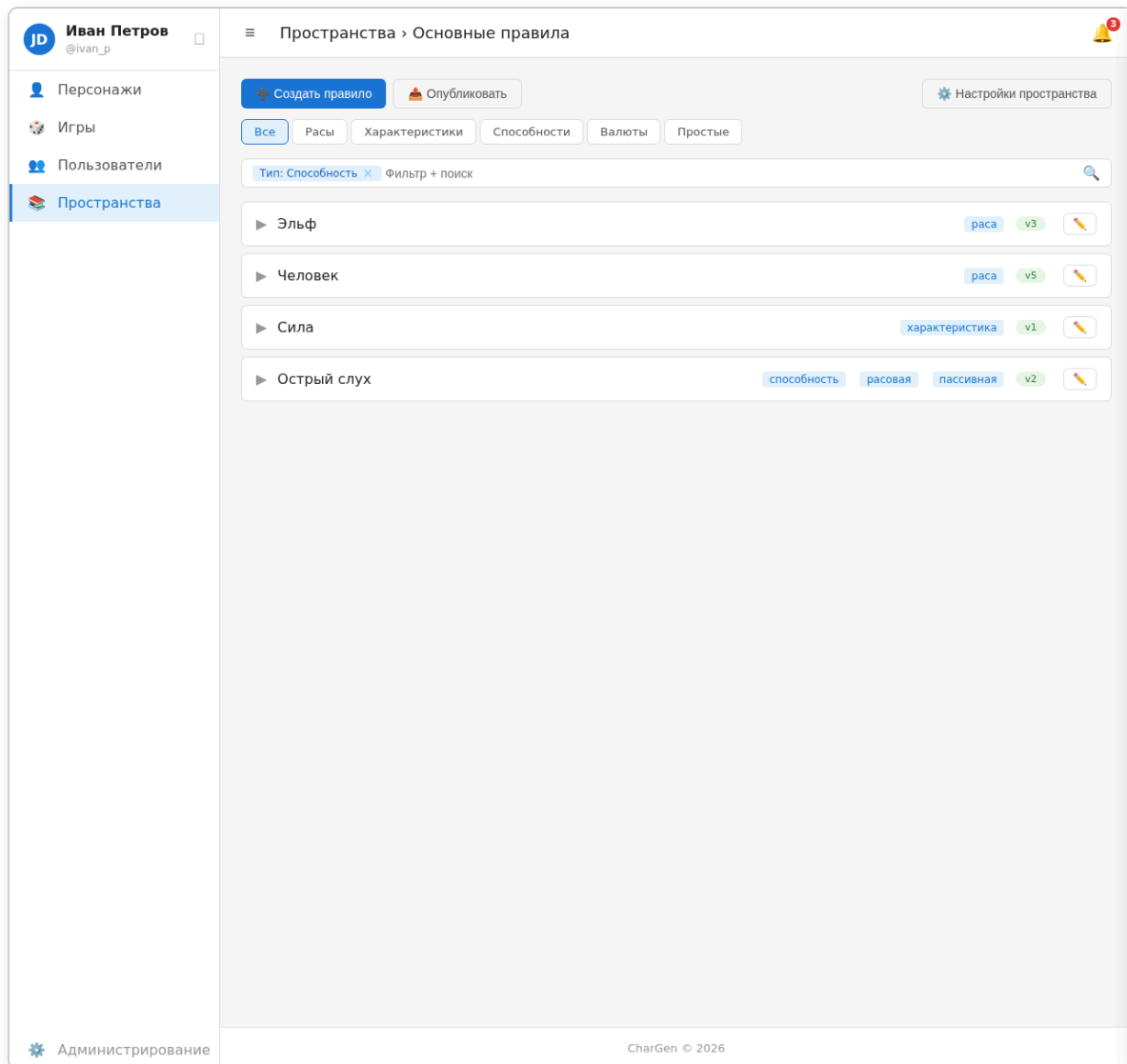
Каждое изменение порождает новую запись `rule_versions(rule_id, space_id, created_at, content...)`. Старые версии остаются. При просмотре пространства на момент `T` показываются самые новые версии правил с `created_at <= T` в этом пространстве.

Пространства — наследование

✓ Выбрано: копирование снимка (snapshot copy)

- При наследовании пространство `B` от `A` в момент `T`: все актуальные версии правил из `A` копируются в `B` с теми же `created_at`.
- После копирования `B` полностью независим от `A`.
- Наследование может быть цепочкой ($A \rightarrow B \rightarrow C$). Каждое копирование независимо — цепочка не создаёт дополнительной сложности.
- Простота запросов: `WHERE space_id = ?` без дополнительных условий.





Публикация правил между пространствами

- У каждого правила глобальный UUID, единый для всех пространств.
- Публикация создаёт новую версию того же `rule_id` в целевом пространстве.
- При публикации показывается diff: последняя версия в целевом пространстве → публикуемая версия.
- Если в целевом пространстве правило правил вручную — diff покажет расхождение, админ видит это.

Удаление правил

✓ Маркер-версия

Создаётся новая запись в `rule_versions` с `active = false`. Все поля, кроме `active`, — такие же, как в последней активной версии (копия). `created_at` = время удаления.

При просмотре пространства на момент T: последняя версия правила с `active = false` означает, что правило удалено. Удаление — частный случай версии, единообразно с soft-delete других сущностей.

Права доступа

Система прав детализирована в Этапе 6. Краткая сводка:

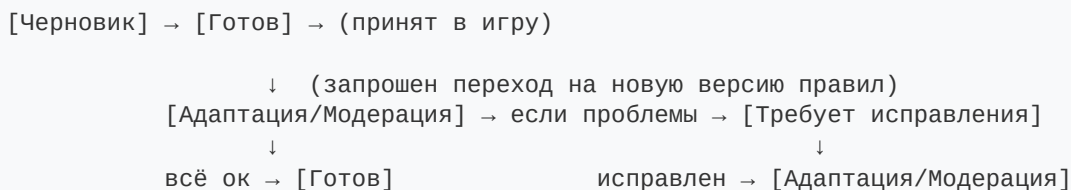
- **Пользователи:** `user.view`, `user.view_sensitive`, `user.create`, `user.edit`, `user.deactivate`.
- **Персонажи:** `character.create`.
- **Группы:** `group.view`, `group.create`, `group.edit`, `group.deactivate`.
- **Игры:** `game.create`, `game.view_all`, `game.edit_all` (глобальные); `game.edit`, `game.moderate`, `game.manage` (per-game).
- **Пространства:** `space.create`, `space.view_all`, `space.edit_all` (глобальные); `space.view`, `space.comment`, `space.edit` (per-space).
- Новые ключи добавляются без миграции схемы.

Параллельное редактирование

- Без специального механизма синхронизации.
- Изменения — одна транзакция. Если хотя бы одно правило конфликтует — не сохраняется **ни одно**.
- Пользователю показываются все конфликты (diff старой / своей / чужой версии) и даётся выбор по каждому: оставить чужую / оставить свою / редактировать.
- Пакет правил может быть логически связан для администратора — решение по всей пачке принимается вместе.

Персонаж: жизненный цикл

Состояния



(детали — см. Этап 4, раздел «Статусы персонажа»)

Привязка к игре

- Владелец персонажа может инициировать переход на новую версию правил в любой момент.
- Если персонаж в игре — **старая версия остаётся в игре** (заморожена, read-only).
- Ведущий может перевести игру на новую версию правил:
 - Персонажи игры становятся неактивными.
 - Игроки обновляют персонажей → новая версия проходит проверку.
 - Если валидна — замещает старого персонажа в игре.
 - Если нет — попадает на модерацию.
- При переходе на новую версию образуются две сущности: P_v1 (старый, в игре) и P_v2 (новый, проходит модерацию). После одобрения P_v2 замещает P_v1.

Очистка истории

Решено: не делать. Удаление старых версий правил может сломать ссылки от персонажей. Если в будущем возникнет необходимость — добавится `archived_at` (soft-delete).

Конец Этапа 1.

Этап 2: Согласованные решения

Ключевое понимание

В разных версиях правил может быть разный набор типов и валют. Например, старая версия не имела ОЛ — значит и экран покупок за ОЛ не показывается. Это значит, что система должна быть версионно-осведомлённой: не только контент правил версионится, но и сам набор того, «какие типы существуют в этой версии», должен определяться данными, а не хардкодом.

Перегруппировка

А. Валюты — отдельная сущность, но часть контента версии

Сущность «Валюта»:

- **Системное имя** (machine key) — внутренний идентификатор (`os` , `ol` , `or` , `ou` , `ov`). Используется движком для привязки стоимостей и расчётов.
- **Основное имя** — например «Очки Создания»
- **Варианты имени** (aliases) — массив, например [`"ОС"` , `"Очка Создания"`] . Для подсказок в тексте: навёл на «ОС» → всплывает «Очки Создания — валюта, используемая при создании персонажа.»
- **Описание** — текст, отображаемый в подсказке

Базовые значения **не хранятся** в самой валюте. ОС и ОР определяются игрой. ОЛ вычисляется правилами возраста (отдельное правило с формулой). Количество валюты у персонажа — это данные персонажа, не настройка валюты.

Валюты — **версионизуемые правила** (имеют `rule_id` , версии в пространствах). Разные версии правил могут иметь разный набор валют.

Б. Типы правил

Все типы — правила: версионизируются, имеют глобальный ID, хранятся в `rule_versions` .

Базовые поля всех правил: `name` , `description` (HTML), `spec` (JSON-блоки — «Спецификация», структурированные данные), `keywords` (теги). Ниже указаны только типоспецифичные поля.

Тип	Описание
Простое правило	Только база. Никакой дополнительной механики.
Раса	Иерархический контейнер. Определяет характеристики, черты, навыки по умолчанию.
Характеристика	Размерное число [a b] с диапазоном, группой. Объединена с Ресурсом на уровне БД.
Способность	Общий тип. Делится на подтипы через теги.
Предмет	Именованная сущность с категорией, стоимостью, весом, прочностью. Подтипы: деньги, снаряжение (оружие/броня/щит), прочее.
Эффект	Runtime-статус с уровнями. Правила: наложение, длительность, снятие.
Состояние	Не тип правила. Агрегат runtime-данных (усталость + эффекты + увечья).

Простое правило

Для правил, которые не являются ни навыками, ни чертами, ни характеристиками. Просто текст с ключевыми словами.

- `name`, `description` (HTML), `spec` (JSON-блоки, опционально), `keywords`

Раса

- Иерархия через `parent_race_id` (вид → раса → подвид → ...)
- Дочерняя раса наследует все правила родительской (на уровне контента)
- Версионирование как обычное правило
- Определяет: какие характеристики базовые, какие опционные, какие способности бесплатны/доступны

Характеристика (и Ресурс)

✓ **Объединены на уровне БД** (один тип числового правила). На уровне кода — разные классы.

Поля:

- `is_dimensional: boolean` — размерное или безразмерное
- `min_value, max_value` — диапазон (для характеристик: 3–5, для ресурсов: null)
- `group` — категория для группировки в UI (например, «Физические», «Ментальные», «Магические»)
- `formula: string | null` — для производных характеристик (например, `"min(Реакция, Внимательность)"`). Покрывает все случаи — `base_for` избыточно.

Способность

Не жёсткие подтипы, а категоризация через теги + набор общих полей.

Черты и особенности могут иметь уровни, стоимость, быть отрицательными. Разница между ними — в **контексте использования** (на каком этапе создания доступны, какой валютой оплачиваются). Валюта привязана к этапу:

- Этап «Основа» (ОС) — расы и врождённые черты
- Этап «Личность» (ОЛ) — особенности личности
- Этап «Развитие» (ОР) — навыки и черты

Поля способности:

- `cost: number[]` — стоимость по уровням (массив). Отрицательная стоимость = даёт очки
- `difficulty: number | null` — трудность (для развиваемых: каждый уровень дороже на `difficulty`)
- `levels: number | null` — максимальный уровень (null = без уровней, 1 = не развивается)
- `requirements: Requirement[]` — массив требований
- `grants: Grant[]` — что даёт (характеристики, другие способности, ресурсы, ключевые слова)
- `action_costs: {resource_id: string, amount: number}[] | null` — затраты ресурсов при использовании (например `[{resource_id: "od", amount: 2}]` или `[{resource_id: "qi", amount: 1}, {resource_id: "od", amount: 1}]`)
- `parent_ability_id: uuid | null` — для группировки в UI и цепочек улучшений (навык → улучшение1 → улучшение2). Присутствует у всех способностей (простая способность, заклинание).
- `automatic: boolean` — авто-получение при выполнении условий
- `visibility: string[]` — где отображается: "основа", "личность", "развитие", "игра", "всегда"

Валюта определяется этапом: "основа" → ОС, "личность" → ОЛ, "развитие" → ОР. Указывать валюту отдельно не нужно.

Подтипы — это **теги**: `черта`, `особенность личности`, `навык`, `действие`, `улучшение`, `пассивная`, `множественная`. Пользователь фильтрует по ним.

Предмет

Именованная сущность, предназначенная для хранения в инвентаре персонажа или добычи игры.

Категории (поле `category`):

- `money` — деньги (монеты). Хранятся в граммах меди (gm). Отображение: «5 гз, 7 гс, 9 гм» (1 гз = 10 гс = 100 гм).
- `equipment` — снаряжение. Имеет подтипы (`subtypes: string[]`): `weapon` (оружие), `armor` (броня), `shield` (щит). Один предмет может иметь несколько подтипов (например, `["weapon", "shield"]`).
- `other` — прочее.

Общие поля предмета:

- `cost_gm: int` — стоимость в граммах меди
- `weight: DimensionalNumber | null` — вес (размерное число или null)
- `durability: int | null` — максимальная прочность
- `consumable: bool` — расходуемый

Поля оружия (`weapon`):

- `attacks: Attack[]` — набор атак. Каждая атака:
 - `name: string` — название (например «Рубящий удар»)
 - `range: "melee" | "thrown" | "ranged"` — тип атаки
 - `damage_type: string` — тип урона
 - `damage: DamageFormula` — урон: `{type: "static", value: int}` или `{type: "formula", value: "@{strength}+2"}`
 - `penetration: DamageFormula` — пробитие (аналогично урону)
 - `accuracy: int` — точность (модификатор)

Поля брони (`armor`):

- `defense_slots: [{defense: int, durability: int}]` — набор связок [Защита, Надёжность]

Поля щита (`shield`):

- `block: {efficiency: DimensionalNumber, defense: DimensionalNumber}` — профиль блокирования

Множественные навыки

Хранение:

- Само правило — шаблон: `Владение языком(Язык, x из 3)`
- У персонажа — конкретный экземпляр: `Владение языком: СВИР, уровень 2`
- В будущем — справочники (выбор из списка или свободный ввод)

Контексты отображения правил

Правила отображаются в разных местах сайта, и в каждом месте — свой набор, свой порядок, своя форма:

Контекст	Какие правила	Особенности
Создание персонажа: этап «Основа»	Расы + Черты (ОС)	Фильтр по доступности для расы, группировка, изменение характеристик в реальном времени
Создание персонажа: этап «Личность»	Особенности личности (ОЛ)	Включая отрицательные (дающие ОЛ)
Создание персонажа: этап «Развитие»	Навыки и черты (ОР)	Деревья/ветки навыков, требования, уровни
Карточка персонажа	Все способности персонажа	Текущие уровни, эффекты
Боевая карточка	Действия, ресурсы, эффекты	Компактный вид, только боевое
Документация правил	Все правила	Полный каталог, поиск, фильтры по тегам
Подсказки в тексте	Любые (по ссылке/ ховеру)	Всплывающее окно с именем и описанием

Для каждого контекста нужна настраиваемая сортировка и возможность выбрать, какие блоки описания показывать.

Теги

✓ Теги — плоский справочник, единый для всей системы, без версионирования.

```
tags(id, string_id, name, description, active)
```

- `string_id` — уникальный строковый идентификатор (`"melee"` , `"magic"` , `"stealth"`)
- `name` — отображаемое имя («Ближний бой», «Магия», «Скрытность»)
- `description` — описание (необязательно)
- `active` — soft-delete (тег скрывается из выбора, старые связи сохраняются)

Используются для:

- Фильтрации и поиска правил
- Группировки способностей в UI
- Условных бонусов (`+1 к Восприятию, если владеете двумя навыками с тегом X`)

- Определения подтипов (черта, навык, действие и т.д.)

Привязка к правилу: `rule_tags(rule_version_id, tag_id)`. Конкретная версия правила имеет определённый набор тегов. При создании новой версии набор тегов может меняться.

Варианты реализации: **плоский список** (без иерархии). Иерархию добавить при необходимости.

Система требований

```
{
  "type": "has_ability" | "has_ability_level" | "has_tag" | "characteristic_val
  "params": { ... },
  "children": [ ... ] // для and/or
}
```

Описание и Спецификация

Правило имеет два вида контента:

1. **Описание** — простой HTML (WYSIWYG-редактор). Человекочитаемый текст с форматированием.
2. **Спецификация** — JSON-блоки структурированных данных (механика правила, не отображаемая напрямую):

```
{
  "blocks": [
    { "type": "text", "content": "..." },
    { "type": "cost_table", "levels": [1, 2, 3], "difficulty": 1 },
    { "type": "requirements" },
    { "type": "action_cost", "value": 2, "unit": "ОД" },
    { "type": "level_scaling", "levels": [
      { "level": 1, "effect": "...", "cost": 1 },
      { "level": 2, "effect": "...", "cost": 2 }
    ] }
  ]
}
```

Согласованные решения по Этапу 2

1.  **Валюты** — отдельная сущность, но версионизируемая.
2.  **Раса** — тип-контейнер с `parent_race_id` для иерархии.

3.  **Характеристики + Ресурсы** — общий тип на уровне БД, разные классы в коде.
4.  **Способности** — общий тип, подтипы через теги. `parent_ability_id` для группировки — у всех способностей (не только улучшений).
5.  **Эффекты** — **отложены**, требуют детальной проработки.
6.  **Состояние** — не тип правила, runtime-агрегат.
7.  **Простое правило** — минимальный тип (название, HTML-описание, теги).
8.  **Теги** — плоский справочник, глобальный для системы, без версионирования.
`(id, string_id, name, description, active)`.
9.  **Множественные навыки** — шаблон + экземпляр у персонажа.
10.  **Описание** — простой HTML (WYSIWYG). **Спецификация** — JSON-блоки структурированных данных.
11.  **Требования** — единая модель с типизированными условиями.
12.  **Теги как способ определения подтипов** способностей (черта, навык, действие и т.д.)

Конец Этапа 2.

Этап 3: Размерные числа —

- ✓ **Хранение:** `value INT, size INT`, для безразмерных — `size = NULL`.
 - ✓ **Операции:** PHP-класс `DimensionalNumber` с `add`, `sub`, `step`, `compareTo`.
 - ✓ **Коррекция диапазона:** при выходе за `[3, 5]` — смена размера, значение сбрасывается к границе.
 - ✓ **Сравнение:** приводить к **меньшему** размеру.
 - ✓ **Отображение:** стрелочная нотация. При `>1` стрелке — степень (`↑7`). Компонент с поддержкой ввода.
 - ✓ **Округление:** `floor` для не-промежуточных результатов.
 - ✓ **Размерные + безразмерные:** не складываются.
-

Конец Этапа 3.

Этап 4: Создание персонажа

Макет: Создание персонажа (/characters/new)

The screenshot shows a web interface for creating a new character. On the left is a sidebar with the user's profile 'Иван Петров @ivan_p' and navigation links for 'Персонажи' (Characters), 'Игры' (Games), 'Пользователи' (Users), and 'Пространства' (Spaces). The main area is titled 'Создание персонажа' (Character Creation) and contains a form for a 'Новый персонаж' (New Character), which is currently a 'черновик' (draft). The form has two main sections: 'Выберите способ создания персонажа' (Choose a way to create a character) with radio buttons for 'Свободное создание' (Free creation) and 'Через игру' (Through game), and 'Бюджет персонажа' (Character budget) with input fields for 'ОС (Очки создания)' (Creation points) and 'ОР (Очки развития)' (Development points). A 'Пространство правил' (Rules space) dropdown is also present. At the bottom right are 'Отмена' (Cancel) and 'Далее' (Next) buttons. The footer includes 'Администрирование' (Administration) and 'CharGen © 2026'.

Этапы создания

1. Выбор расы

- Если создание через Игру — ОС и ОР определяются игрой. Если свободное — можно задать или оставить пустыми.
- Первым делом — выбор расы (вида и подвида).
- Карточка расы: полная информация (характеристики, черты, особенности).
- Итем расы в списке: краткое описание.
- Фильтры: по виду, по тегам, по названию.
- **Раса тратит ОС из бюджета игры, а не определяет его.** Значение ОС в профиле расы — это её стоимость (отрицательная стоимость = даёт ОС). Бюджет ОС задаётся игрой или вручную.

2. Распределение ОС (этап «Основа»)

- Тратятся на врождённые черты.
- Черты имеют фильтры: **доступные** (можно взять сейчас), **недоступные** (с указанием причины), **расовые** (открываются расой), **общедоступные** (без расовых ограничений).
- Во время выбора — live-изменение характеристик.
- Можно потратить больше лимита → персонаж в статусе «Черновик».
- **Из документов:** у каждой расы профиль (ОС, характеристики, бесплатные черты, опции).

3. Особенности личности (этап «Личность», ОЛ)

- После траты ОС.
- Могут давать способности.
- Могут быть отрицательными (давать ОЛ, а не тратить).
- **Не во всех версиях правил есть этот этап** — если ОЛ нет, пропускается.

4. Способности за ОР (этап «Развитие»)

- После траты ОЛ (или после ОС, если ОЛ нет).
- Навыки, черты, действия — всё, что покупается за ОР.
- Деревья/ветки навыков, требования.

5. Закупка (инвентарь)

- После распределения ОР.
- Бюджет в валюте «Деньги» (ключ **money**) — задаётся игрой или вручную.
- Список доступных предметов из правил пространства, фильтр по категории.
- Предметы добавляются в инвентарь персонажа с указанием количества.
- Остаток бюджета конвертируется в монеты в инвентаре (формат «5 гз, 7 гс, 9 гм»).

6. Сохранение

- **Черновик:** в любой момент, без проверки требований, даже при перетрате лимитов.
- **Готов:** только при соблюдении лимитов и всех требований.
- **История персонажа** — отдельный этап, позже.

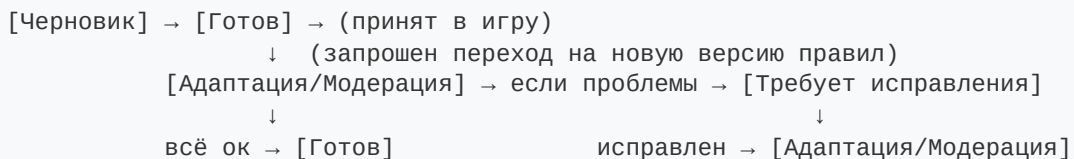
7. Редактирование после создания — общий случай

- В любой момент. Каждое изменение → новая версия (copy-on-write).
- Вне игры — без чужого подтверждения.

8. Редактирование в игре — сессионная модель

- Игрок правит персонажа (инвентарь, характеристики, способности) — все изменения **накапливаются в автосохраняемом черновике**.
- Черновик сохраняется непрерывно (по каждому изменению + периодически). Не теряется при закрытии браузера.
- «Живой» персонаж в игре остаётся неизменным, пока черновик не отправлен на модерацию.
- По окончании сессии (или в любой момент) игрок нажимает **«Отправить на модерацию»** → черновик становится новой версией, уходит ведущему.
- Ведущий утверждает все изменения разом или отправляет на доработку.
- **Ведущий** имеет право `game.edit_inventory` — редактировать инвентарь персонажей игры напрямую (без модерации).

Статусы персонажа (из Этапа 1)



Редактирование персонажа

- Любое изменение создаёт новую версию персонажа (версионирование, как у правил).
- Можно откатиться к предыдущей версии.
- При переходе на новую версию правил:
 - Старый персонаж остаётся в игре (если он в игре).
 - Новая версия проходит валидацию.
 - Если не проходит — на модерацию к ведущему.

JD

Иван Петров

@ivan_p

Персонажи

Игры

Пользователи

Пространства

Администрирование

Редактирование: Элиандра

3

Черновик

Готово

Эльф

Стоимость: 10 ОС

Врождённые черты

ОС: 14 / 20

Личность

0 / 3 ОП

Развитие

5 / 15 ОП

Инвентарь

не скоро

Сила: 3

Ловкость: 4

Выносливость: 3

Восприятие: 4

Интеллект: 2

Мудрость: 2

Все

Доступные

Недоступные

Расовые

Общедоступные

Тип: расовые

Фильтр + поиск

+2 ОС

Острый слух

расовая

пассивная

-

1

+

2 ОС

Ночное зрение

расовая

-

1

+

1 ОС

Лёгкая походка

расовая

пассивная

-

0

+

3 ОС

Магическая интуиция

магическая

-

0

+

4 ОС

Крепкое телосложение

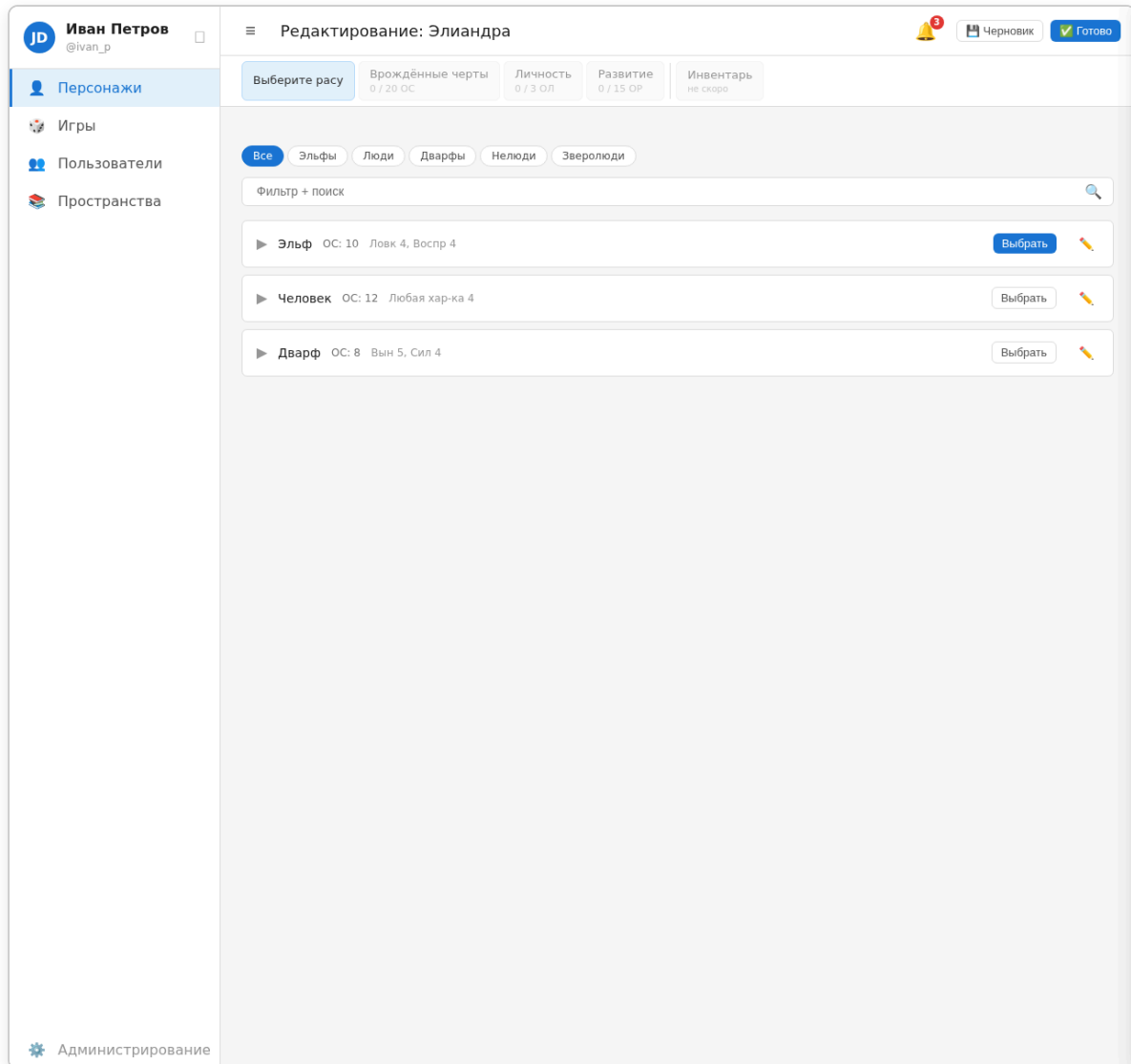
доступная

пассивная

-

0

+



Вопросы для обсуждения

1. **Live-изменение характеристик** при выборе черт — как технически реализовать пересчёт на фронте? Пересчитывать всё на каждый клик или кешировать промежуточные состояния?
2. **Перетрата лимита** — если игрок потратил больше ОС, чем есть — персонаж черновик. А если потом убрал лишнее — можно ли снова сделать готовым? Автоматически или требуется подтверждение?
3. **Будущие расширения** (Закупка, История) — как заложить гибкость уже сейчас?
4. **Отрицательные ОЛ** — если особенность даёт +2 ОЛ, а игрок их не тратит — что происходит? ОЛ сгорают или остаются?
5. **Свободное создание** — если ОС и ОП не заданы, то лимитов нет? Персонаж всегда черновик?

Согласованные решения

Архитектура расчётов

✓ Фронт — активные расчёты, бэк — только валидация.

- При каждом выборе/изменении на фронте — минимальный пересчёт (только то, что изменилось).
- При сохранении бэк проверяет: лимиты, требования, соответствие версии правил.
- Черновик сохраняется без валидации в любом состоянии.

Фильтры черт при выборе

✓ Четыре категории: доступные, недоступные (с причиной), расовые, общедоступные.

Редактирование персонажа — copy-on-write

✓ При редактировании готового персонажа создаётся его копия-черновик. Оригинал остаётся нетронутым. Когда игрок нажимает «Готово» — копия проверяется и, если всё ок, подменяет оригинал.

Этот же механизм работает для перехода между версиями правил: старая версия P_v1 остаётся в игре, новая P_v2 = копия-черновик, которая проходит проверку и затем подменяет P_v1.

JD

Иван Петров

@ivan_p

Персонажи

Игры

Пользователи

Пространства

Администрирование

Редактирование: Элиандра

3

Черновик

Готово

Эльф

Врождённые черты

Личность

Развитие

Инвентарь

Сила: 3

Ловкость: 4

Выносливость: 3

Восприятие: 4

Интеллект: 2

Мудрость: 2

Все

Доступные

Недоступные

Расовые

Общедоступные

Тип: расовые

Фильтр + поиск

+2 ОС

Острый слух

расовая

пассивная

-

1

+

2 ОС

Ночное зрение

расовая

-

1

+

1 ОС

Лёгкая походка

расовая

пассивная

-

0

+

3 ОС

Магическая интуиция

магическая

-

0

+

4 ОС

Крепкое телосложение

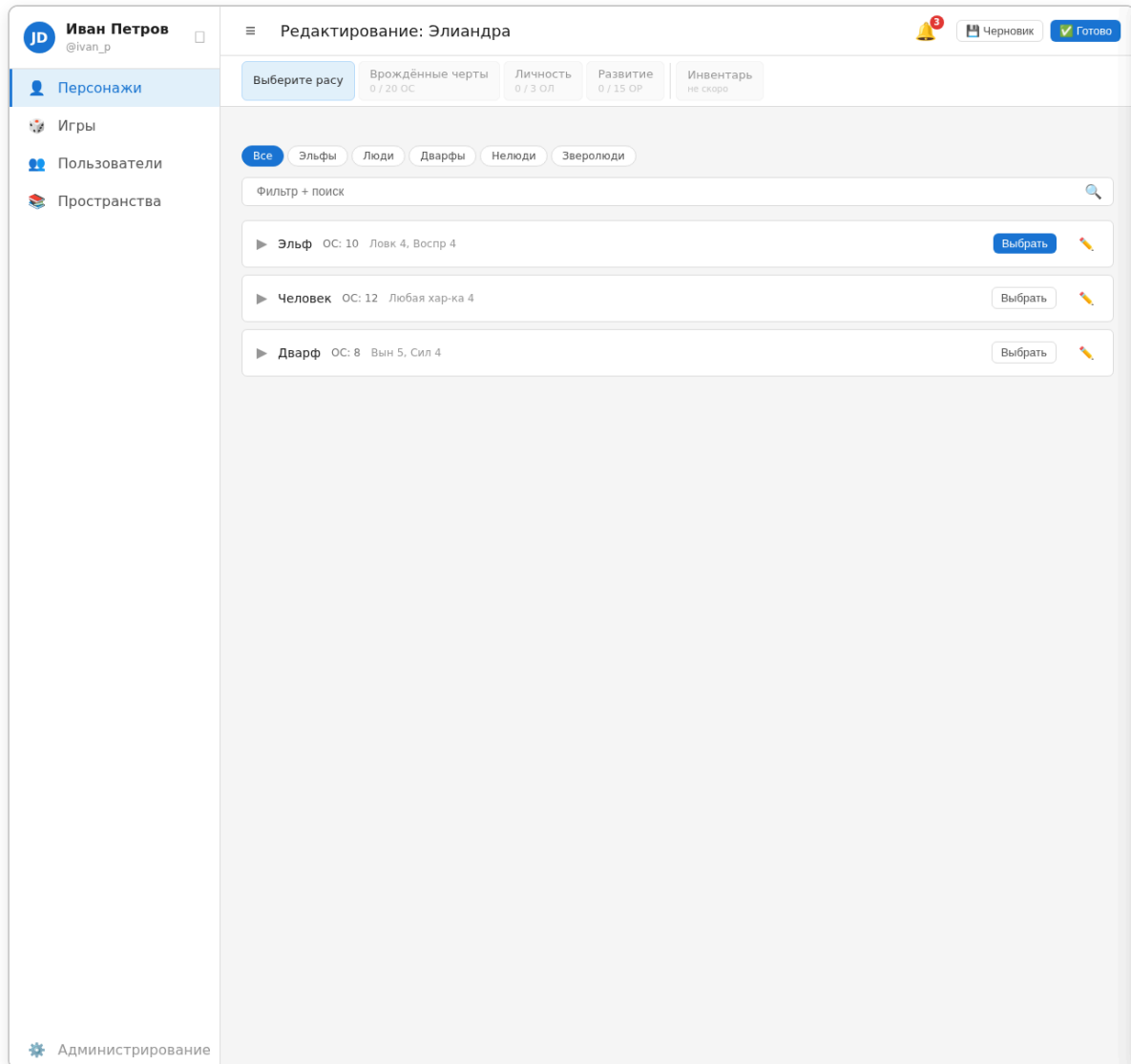
доступная

пассивная

-

0

+



Статусы (уточнение)

- **Черновик** — не снимается автоматически. Нужно нажать кнопку «Готов».
- **Готов** — персонаж соответствует всем требованиям.
- Для входа в игру — требуется одобрение ведущего.
- Вне игры — изменения без чужого подтверждения.
- **В игре** — изменения накапливаются в автосохраняемом черновике (сессионная модель). По окончании сессии игрок отправляет изменения на модерацию ведущему.

Редактирование в игре — сессионная модель

✓ **Сессионная модель:** игрок редактирует персонажа в игре — изменения пишутся в автосохраняемый черновик. «Живой» персонаж не меняется. По окончании сессии (или в любой момент) игрок отправляет черновик на модерацию. Ведущий утверждает все изменения разом или отправляет на доработку.

✓ **Ведущий** может напрямую редактировать инвентарь персонажей игры (право `game.edit_inventory`).

Отрицательные ОЛ

✓ Неиспользованные ОЛ **сгорают**. Если особенность дала +2 ОЛ, а игрок их не потратил — они пропадают.

Лимиты при свободном создании

✓ Если лимиты ОС/ОР не заданы — персонажа можно сделать готовым в любой момент, при условии корректности всех расчётов и соответствия версии правил.

Будущие расширения (контекст)

- **История персонажа** — имя, предыстория + хронологический лог сообщений.
- При проектировании архитектуры БД и кода учитывать возможность добавления новых этапов создания.

Конец Этапа 4.

Этап 5: Скрытые проблемы

1. Производительность запросов «на момент времени T»

Проблема: Каждый просмотр пространства требует для каждого правила найти `MAX(created_at) <= T`.

С ростом числа версий (сотни тысяч) такие запросы могут стать медленными.

Возможные решения:

- Композитный индекс `(space_id, rule_id, created_at DESC)` — покрывает основной запрос
- Таблица `current_rules(space_id, rule_id, version_id)` — материализованное представление последней активной версии каждого правила. Обновляется при каждом подтверждении изменений. Для запросов «на момент T» — падаем на основную таблицу
- Партиционирование по `space_id` или по датам

2. Наследование пространств — операция копирования

Проблема: При наследовании большого пространства копирование всех правил может быть долгой операцией. Админ ждёт.

Решение: делать асинхронно (job/queue). Во время копирования пространство помечается как «наследуется», правила постепенно добавляются. UI показывает прогресс.

3. Циклические зависимости в характеристиках

Проблема: Производные характеристики могут образовать цикл: $A = f(B)$, $B = f(C)$, $C = f(A)$. При пересчёте — бесконечный цикл или некорректный результат.

Решение: детектор циклов при сохранении правила-характеристики. Максимальная глубина вычислений с ошибкой «слишком глубоко».

4. Лавинная перепроверка персонажей при смене версии правил

Проблема: Если в игре 50 персонажей, и игра переходит на новую версию правил, нужно проверить каждого. Если синхронно — долго.

Решение: асинхронная очередь задач. Каждому персонажу — отдельная задача валидации. Результаты пишутся в статус персонажа.

5. Copy-on-write — уборка мусора

Проблема: При редактировании создаётся копия-черновик. Если игрок закрыл браузер и не завершил — остаётся «висячий» черновик. Со временем их накапливается много.

Решение:

- Периодическая чистка старых неиспользуемых черновиков (например, старше 30 дней)
- Флаг `draft_of = character_id` для связи, чтобы не дублировать всё подряд
- При начале нового редактирования — переиспользовать существующий черновик или удалять старый

6. Безопасность прав — эскалация привилегий

Проблема: Пользователь с правом `user.edit` и `group.edit` может назначить себе все остальные права.

Решение:

- Права на управление правами должны проверять, что пользователь не может дать себе прав больше, чем у него самого
- Права на пользователей и группы разбиты на отдельные ключи: кто может управлять другими, не может произвольно расширять собственные права

7. Конкурентное создание персонажа в игре

Проблема: Два игрока одновременно создают персонажей в одной игре. Оба тратят одни и те же ресурсы игры.

Решение: блокировка на уровне игры при создании/редактировании персонажа в контексте игры. Или optimistic locking с проверкой ресурсов игры при сохранении.

8. Миграции JSON-блоков Спецификации

Проблема: Описание хранится как JSON-блоки. При добавлении нового типа блока старые версии правил остаются в старом формате. Рендерер должен уметь работать со всеми версиями блоков.

Решение:

- Каждый блок имеет `type` и `version` поля
- Рендерер определяет тип и версию и отображает соответственно
- Миграции данных — отдельные скрипты (не обязательные, рендерер обратно совместим)

9. Архивация старых версий (задел на будущее)

Хотя мы решили не делать очистку истории сейчас, стоит заложить в модель БД возможность пометать версии как `archived` — это позволит в будущем точно архивировать без изменения схемы.

10. Тестирование — критически важно

Система с версионированием, размерными числами, copy-on-write, правилами с требованиями — крайне сложна для ручного тестирования. Потребуется:

- Модульные тесты для `DimensionalNumber` (все операции, граничные случаи)
- Тесты для системы требований (and/or, вложенные условия)
- Тесты для валидации персонажа (mutation-тесты: меняем правило → персонаж должен стать невалидным)
- Тесты для версионности (создание версий, просмотр на момент T)

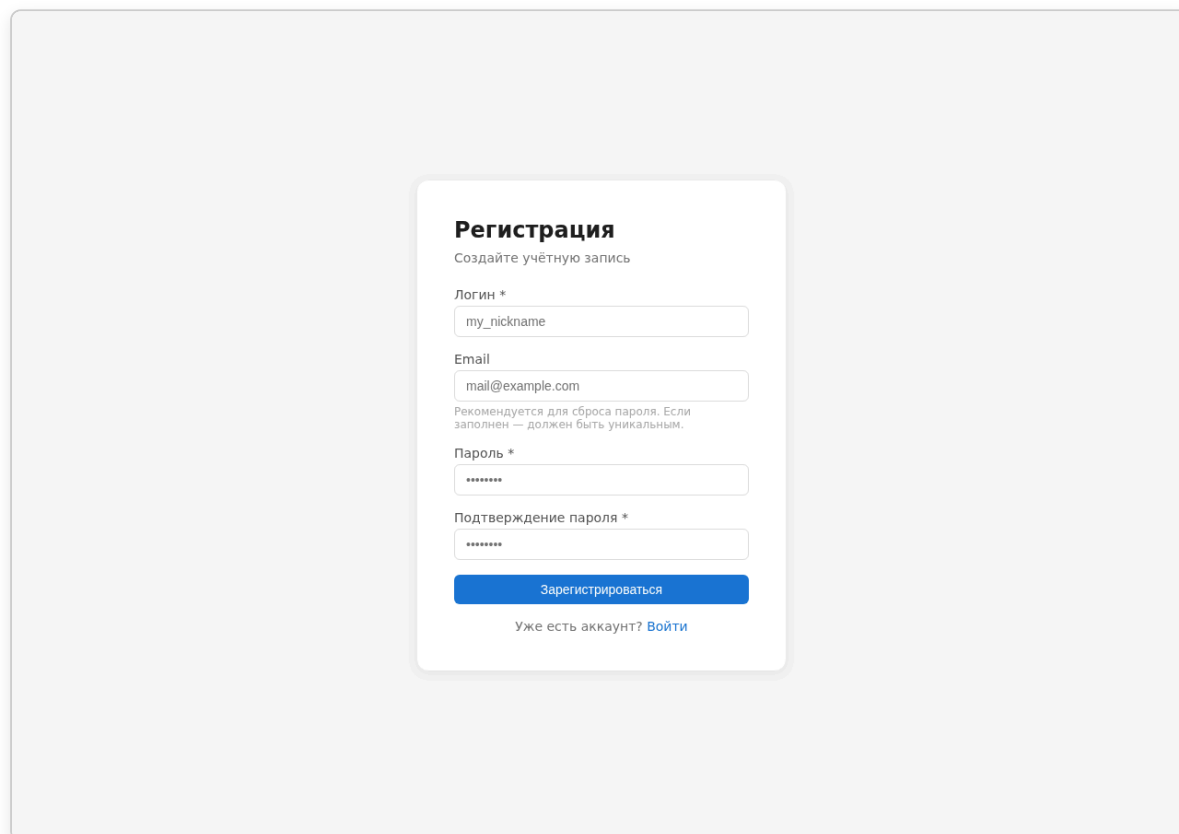
Конец Этапа 5.

Этап 6: Авторизация и управление пользователями

Регистрация

- **Открытая регистрация** — любой желающий может зарегистрироваться.
- По умолчанию новый пользователь получает группу **«Игрок»**.
- Аутентификация: логин + пароль.
- Сброс пароля: стандартный — пользователь вводит email, получает письмо с одноразовой ссылкой, задаёт новый пароль.

Макет: Регистрация (/register)

A mockup of a registration form titled "Регистрация" (Registration) with the subtitle "Создайте учётную запись" (Create an account). The form is centered on a light gray background. It contains four input fields: "Логин *" (Login) with the placeholder "my_nickname", "Email" with the placeholder "mail@example.com", "Пароль *" (Password) with a masked input, and "Подтверждение пароля *" (Confirm password) with a masked input. Below the password fields is a blue button labeled "Зарегистрироваться" (Register). At the bottom, there is a link "Уже есть аккаунт? Войти" (Already have an account? Login).

Модель пользователя

Поле	Обязательное	Описание
login	✓	Уникальный идентификатор
password	✓	Хешируется

Поле	Обязательное	Описание
<code>email</code>	—	Для сброса пароля
<code>first_name</code>	—	Имя
<code>last_name</code>	—	Фамилия
<code>nickname</code>	—	Псевдоним (никнейм)
<code>avatar_file_id</code>	—	Ссылка на файл в <code>files</code>
<code>super_admin</code>	—	Флаг: защищённая учётная запись (нельзя удалить, снять с группы)

Набор полей расширяем (в будущем могут добавляться новые).

Права на пользователей

Права на операции с пользователями разбиты на отдельные ключи (не монолитное `manage_users`):

Ключ	Что даёт
<code>user.view</code>	Видеть список пользователей и чужие профили
<code>user.view_sensitive</code>	Видеть скрытые поля (email, группы)
<code>user.create</code>	Создавать пользователей
<code>user.edit</code>	Редактировать чужие профили
<code>user.deactivate</code>	Деактивировать (банить) пользователей

По умолчанию все пять ключей есть только у группы «Администраторы».

Группа «Игрок» по умолчанию: `user.view`, `character.create`.

Редактирование профиля

- **Пользователь** может менять свой логин, пароль, email, имя, фамилию, псевдоним, аватар. Не может назначать себе группы.
- **Загрузка аватара:** на странице профиля при наведении на аватар снизу появляется полупрозрачная кнопка «Загрузить». Клик открывает системный выбор файла (PNG, JPG, до 2 MB).
- **Пользователь с `user.edit`** может менять всё, включая группы.

JD Иван Петров @ivan_p

Персонажи

Игры

Пользователи

Пространства

Редактирование профиля

JD

PNG, JPG, до 2 MB

Основная информация

Логин

ivan_p

Email

ivan@example.com

Имя

Иван

Фамилия

Петров

Псевдоним (никнейм)

IvanTheGreat

Смена пароля

Текущий пароль

Новый пароль

Сохранить изменения

Группы

Игрок (изменение групп — только с правом 'user.edit')

Администрирование

CharGen © 2026

Деактивация (бан)

- Пользователя может деактивировать только пользователь с правом `user.deactivate`.
- Деактивация — мягкое удаление (soft-delete): пользователь не может войти, данные сохраняются.
- Возможна **временная** деактивация (до определённой даты) с указанием причины.
- Полное удаление не предусмотрено.

Супер-админ

- При установке системы создаётся один **супер-админ**.
- Супер-админ состоит в группе **«Администраторы»**.
- Группа «Администраторы» имеет все права (полный набор `permission_key`).

- **Нельзя:** удалить супер-админа, снять с него группу «Администраторы».
- В остальном группа «Администраторы» — обычная группа: можно добавлять других пользователей (если есть права).

Группы пользователей

- Изначально группы создаёт только **супер-админ**.
- В будущем — пользователь с правом `group.create` может создавать группы.
- **Правило безопасности:** при создании/редактировании группы пользователь не может:
 - назначить группе прав больше, чем имеет сам
 - лишить группу прав, которых сам не имеет
- Пользователь может состоять в нескольких группах, права суммируются.
- Группу можно **деактивировать**: она перестаёт отображаться пользователям и не даёт прав. Полное удаление групп не предусмотрено.
- Назначение прав в UI — группированные чекбоксы по категориям (глобальные, на игры, на пространства).

Права (permission keys)

Права пользователей

Ключ	Что даёт
<code>user.view</code>	Видеть список пользователей и чужие профили
<code>user.view_sensitive</code>	Видеть скрытые поля (email, группы)
<code>user.create</code>	Создавать пользователей
<code>user.edit</code>	Редактировать чужие профили
<code>user.deactivate</code>	Деактивировать (банить) пользователей

Права групп

Ключ	Что даёт
<code>group.view</code>	Просмотр списка групп, состава и прав
<code>group.create</code>	Создание групп
<code>group.edit</code>	Редактирование групп (участники, права)

Ключ	Что даёт
<code>group.deactivate</code>	Деактивация групп

Права пространств (глобальные и per-space)

Ключ	Уровень	Что даёт
<code>space.create</code>	Глобальное	Создание пространств
<code>space.view_all</code>	Глобальное	Просмотр всех пространств
<code>space.edit_all</code>	Глобальное	Редактирование любых пространств
<code>space.view</code>	Per-space	Просмотр пространства
<code>space.edit</code>	Per-space	Редактирование настроек пространства, публикация, деактивация

Права правил (per-space)

Ключ	Уровень	Что даёт
<code>rule.view</code>	Per-space	Просмотр правил в пространстве
<code>rule.create</code>	Per-space	Создание правил
<code>rule.edit</code>	Per-space	Редактирование правил
<code>rule.delete</code>	Per-space	Удаление (деактивация) правил

Права персонажей

Ключ	Что даёт
<code>character.create</code>	Создание персонажей (свободное и в игры)
<code>character.view</code>	Просмотр чужих персонажей (свои всегда видны владельцу)

Права тегов

Ключ	Что даёт
<code>tag.view</code>	Просмотр списка тегов
<code>tag.create</code>	Создание тегов
<code>tag.edit</code>	Редактирование тегов
<code>tag.delete</code>	Удаление (soft-delete) тегов

Права шаблонов уведомлений

Ключ	Что даёт
<code>notification_template.view</code>	Просмотр списка шаблонов
<code>notification_template.create</code>	Создание шаблонов
<code>notification_template.edit</code>	Редактирование шаблонов
<code>notification_template.delete</code>	Удаление шаблонов

Права игр (глобальные и per-game)

Ключ	Уровень	Что даёт
<code>game.create</code>	Глобальное	Создание игр
<code>game.view_all</code>	Глобальное	Просмотр всех игр (включая черновики)
<code>game.edit_all</code>	Глобальное	Редактирование любых игр
<code>game.edit</code>	Per-game	Редактирование настроек игры
<code>game.moderate</code>	Per-game	Модерация пользователей игры
<code>game.manage</code>	Per-game	Управление настройками игры
<code>game.edit_inventory</code>	Per-game	Редактирование инвентаря персонажей игры (для ведущих)

Права чатов

Ключ	Что даёт
<code>chat.create</code>	Создание чатов (приватных, групповых)
<code>chat.message</code>	Отправка сообщений (автоматически у участников чата)
<code>chat.delete</code>	Удаление сообщений/чатов

Все ключи расширяемы — новые добавляются без миграции схемы.

Статусы игры

Статус	Видимость
Черновик	Только создатель и пользователи с <code>game.view_all</code>
Открытая	Видна всем, вступить можно без приглашения

Статус	Видимость
Закрытая для просмотра	Видна по приглашению + участникам
Закрытая для вступления	Видна всем, вступить только по приглашению

Роли в игре

- **Владелец** — создатель игры, всегда ведущий. Может приглашать других ведущих.
- **Ведущий** — пользователь с правами на редактирование и модерацию в игре (по решению владельца).
- **Участник** — игрок, чей персонаж в игре. Может получить дополнительные права от владельца (индивидуально).
- Владелец может выдавать участникам индивидуальные права: редактирование игры, модерация пользователей и т.д.

Модель выдачи прав на объекты

Права на игру или пространство выдаются двумя способами (работают одновременно):

1. **Через группы:** группе назначаются права на объект → все участники группы получают эти права.
2. **Индивидуально:** конкретному пользователю выдаётся право на объект.

Права суммируются: если пользователь получил `game.edit` через группу и `game.moderate` индивидуально — у него есть оба.

Визуальная модель

Глобальные права (группа пользователей) → что можно в системе

- └─ Права на пространство (группа + индивидуально)
- └─ Права на игру (статус + роль + группа + индивидуально)

Система авторизации

Принцип

Авторизация — middleware-слой, проверяющий доступ перед каждым действием. Два уровня проверки:

1. **Глобальные права** — проверяются по `group_permissions` для всех групп пользователя. Если есть — доступ есть.
2. **Per-object права** — если запрос касается конкретного объекта (игра, пространство), дополнительно проверяются права на этот объект.

Super-admin bypass

Пользователь с `super_admin = true` **обходит все проверки** прав. Middleware проверяет это первым делом — если супер-админ, сразу пропускать.

Правило «свой vs чужой»

Для ресурсов, где пользователь является владельцем (свой профиль, свой персонаж), **глобальное право не требуется** — только аутентификация. Глобальное право нужно для действий над **чужими** объектами.

Конкретно:

Ресурс	Свой	Чужой
Профиль (<code>/users/:id</code>)	Всегда доступен для редактирования владельцем	<code>user.edit</code>
Персонаж (<code>/characters/:id</code>)	Всегда доступен владельцу	<code>character.view</code> *
Пространство (<code>/spaces/:id</code>)	— (нет владельца в этом смысле)	<code>space.view</code> / <code>space.edit</code>
Игра (<code>/games/:id</code>)	Владелец и ведущие — полный доступ	По статусу игры + <code>game.edit_all</code>
Правило (<code>/spaces/:id/rules/:ruleId</code>)	—	<code>space.view</code> / <code>space.edit</code>
Группа (<code>/admin/groups/:id</code>)	—	<code>group.*</code>
Теги (<code>/admin/tags</code>)	—	<code>tag.*</code>
Шаблоны уведомлений	—	<code>notification_template.*</code>

* `character.view` — новый ключ. По умолчанию персонажи видны только владельцу и участникам игры.

Алгоритм проверки (псевдокод)

```
function checkAccess(user, action, resourceType, resourceId = null):
    // 1. Super-admin bypass
    if user.super_admin: return true

    // 2. Собрать все права пользователя
    permissions = globalPermissions(user) // из group_permissions
    if resourceId:
        permissions += objectPermissions(user, resourceType, resourceId)

    // 3. Проверить специфичное право
    if action in permissions: return true

    // 4. Проверить право «своего» объекта
    if isOwner(user, resourceType, resourceId): return true

    // 5. Доступ запрещён
    return false
```

Связка «роль в игре + права»

Роль `owner` даёт `game.edit`, `game.moderate`, `game.manage` на эту игру без записи в `game_member_permissions`. Роль `gm` даёт `game.edit`, `game.moderate`. Роль `player` не даёт ничего сверх прав группы.

При проверке доступа к игре: если роль даёт нужное право — доступ есть, даже если в `game_member_permissions` нет соответствующей записи.

Множественные запросы

Если страница отображает несколько сущностей (список персонажей, список игр), **не делается N запросов прав**. Вместо этого:

- **Глобальные права** проверяются один раз на middleware
- **Список доступных объектов** фильтруется одним SQL-запросом с JOIN на `game_members`, `space_permissions`, `game_member_permissions` и т.д.
- Для каждого объекта на фронт передаётся computed-флаг `can_edit`, `can_delete` и т.д.

Сводка решений Этапа 6

1.  **Открытая регистрация**, новый пользователь — группа «Игрок».
2.  **Поля**: логин (уникальный, обязательный), email (для сброса пароля), пароль.
3.  **Супер-админ**: один при установке, в группе «Администраторы». Нельзя удалить или снять с неё.
4.  **Группы**: изначально создаёт супер-админ. В будущем — с правом `group.create`. Нельзя дать прав больше, чем у себя.

5.  **Права на объекты:** группы + индивидуально. Суммируются.
 6.  **Статусы игры:** Черновик / Открытая / Закрытая для просмотра / Закрытая для вступления.
 7.  **Роли в игре:** Владелец → Ведущий → Участник. Владелец выдаёт права индивидуально.
 8.  **Единый формат ключей:** `объект.действие ( user.* , group.* , game.* , space.* , character.* , tag.* , notification_template.* )`.
 9.  **Authorization middleware:** super-admin bypass, двухуровневая проверка (глобальные + per-object), правило «свой vs чужой» (свой профиль/персонаж — без глобального права).
 10.  **Роль в игре:** `owner` = полный доступ, `gm` = edit + moderate, `player` = только права группы. Роль даёт права без записи в `game_member_permissions`.
 11.  **Множественные запросы:** фильтрация через JOIN, не N отдельных проверок.
 12.  **`character.view`** — новый ключ для просмотра чужих персонажей.
-

Конец Этапа 6.

Этап 7: Интерфейс — структура разделов

Аутентификация

Страницы входа/регистрации — отдельные, вне основного лейаута (без сайдбара, минималистичный дизайн).

Путь	Страница	Доступ
<code>/login</code>	Вход. Поля: логин/email, пароль. Кнопка «Запомнить меня» (продлённая сессия). Ссылка «Забыли пароль?».	Неавторизованные
<code>/register</code>	Регистрация. Поля: логин (уникальный), email (опционально, но рекомендуется для сброса пароля), пароль, подтверждение пароля. Email должен быть уникальным, если заполнен. После регистрации — сразу вход.	Неавторизованные
<code>/forgot-password</code>	Запрос сброса пароля. Поля: логин или email. Система находит пользователя по логину (приоритет) или email. Если найден — отправляет письмо на привязанный email (если он есть); если email не указан — выводит сообщение «Для этого аккаунта не указан email, обратитесь к администратору».	Неавторизованные
<code>/reset-password/:token</code>	Сброс пароля. Пароль, подтверждение. Token одноразовый, expires_at.	Неавторизованные
<code>/logout</code>	Выход. POST-запрос, аннулирует сессию, редирект на <code>/login</code> .	Авторизованные

Макет: Вход в систему (/login)

CharGen

Войдите в систему управления персонажами

Логин или email

login@example.com

Пароль

☐ Запомнить меня

Войти

[Забыли пароль?](#) · [Регистрация](#)

Макет: Регистрация (/register)

Регистрация

Создайте учётную запись

Логин *

my_nickname

Email

mail@example.com

Рекомендуется для сброса пароля. Если заполнен — должен быть уникальным.

Пароль *

Подтверждение пароля *

Зарегистрироваться

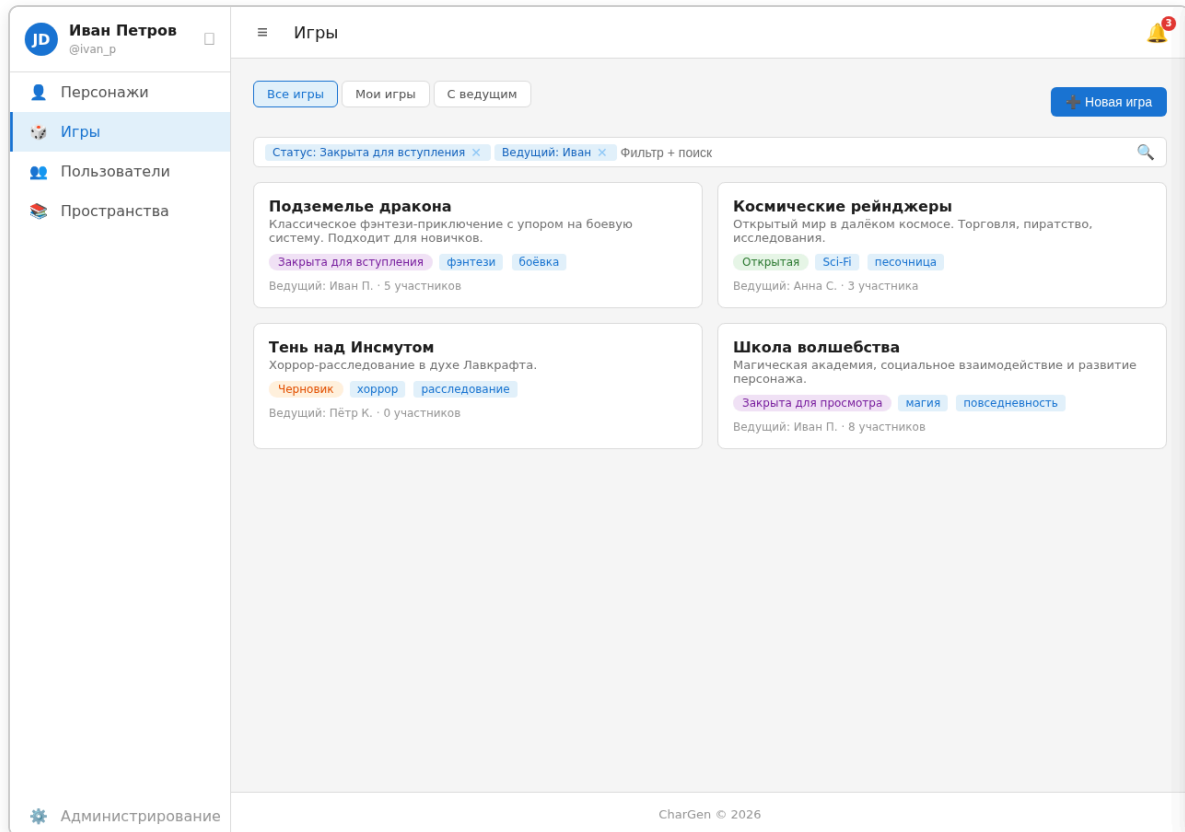
Уже есть аккаунт? [Войти](#)

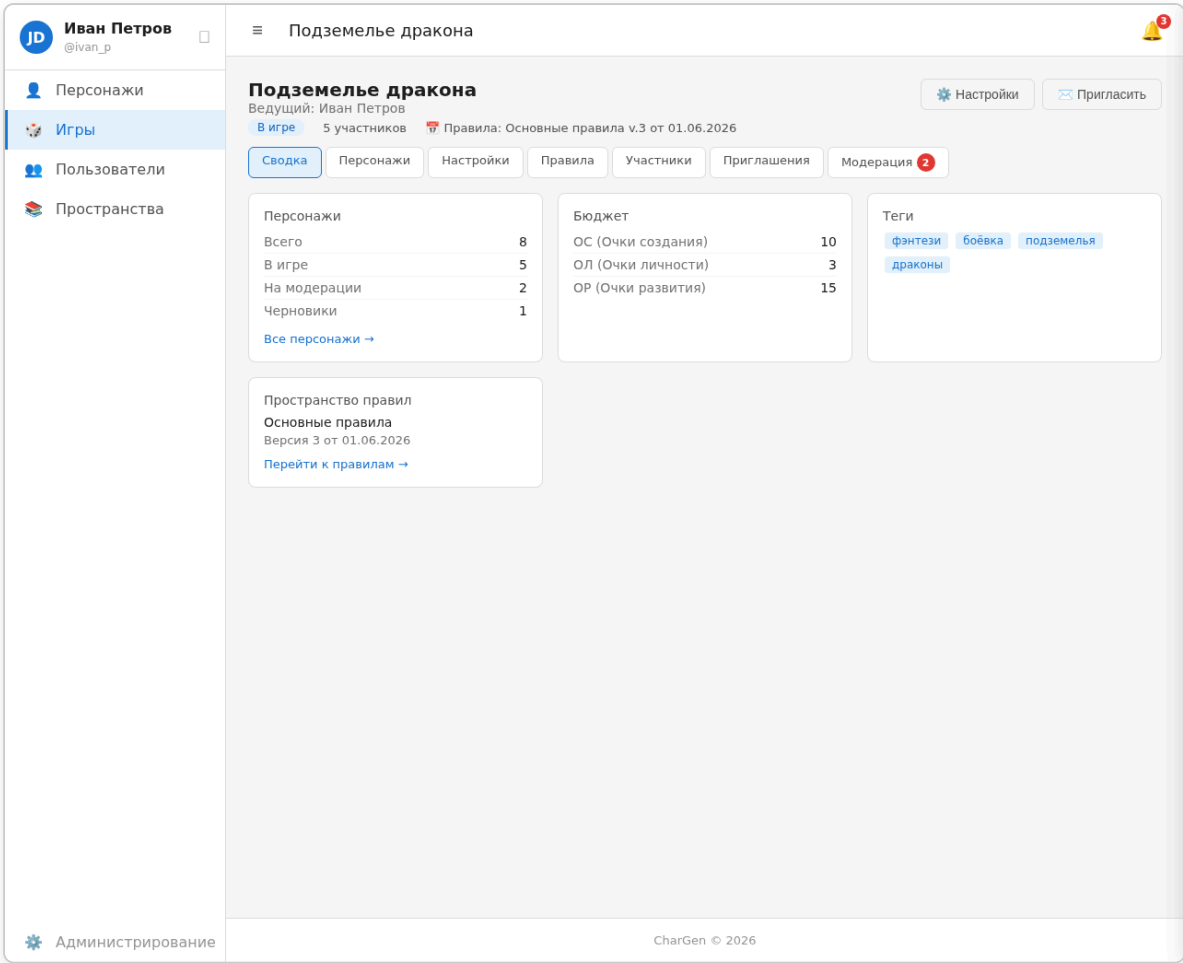
The mockup shows a centered white card on a light gray background. The card has a title 'Сброс пароля' in bold. Below it is a paragraph explaining the process: 'Введите логин или email. Если email привязан к аккаунту — на него придёт ссылка для сброса.' There is a text input field with the placeholder 'ivan_p или ivan@example.com'. Below the input is a blue button labeled 'Отправить ссылку'. At the bottom of the card is a blue link 'Вернуться ко входу'.

Сессии

- После входа создаётся запись в `sessions`, токен сохраняется в httpOnly cookie.
- Токен — случайная строка, хранится хешированной в БД.
- Время жизни: по умолчанию 24ч, «Запомнить меня» — 30 дней.
- При каждом запросе middleware проверяет токен из cookie, подгружает пользователя.
- При выходе сессия удаляется из БД.

Раздел «Игры» (/games)



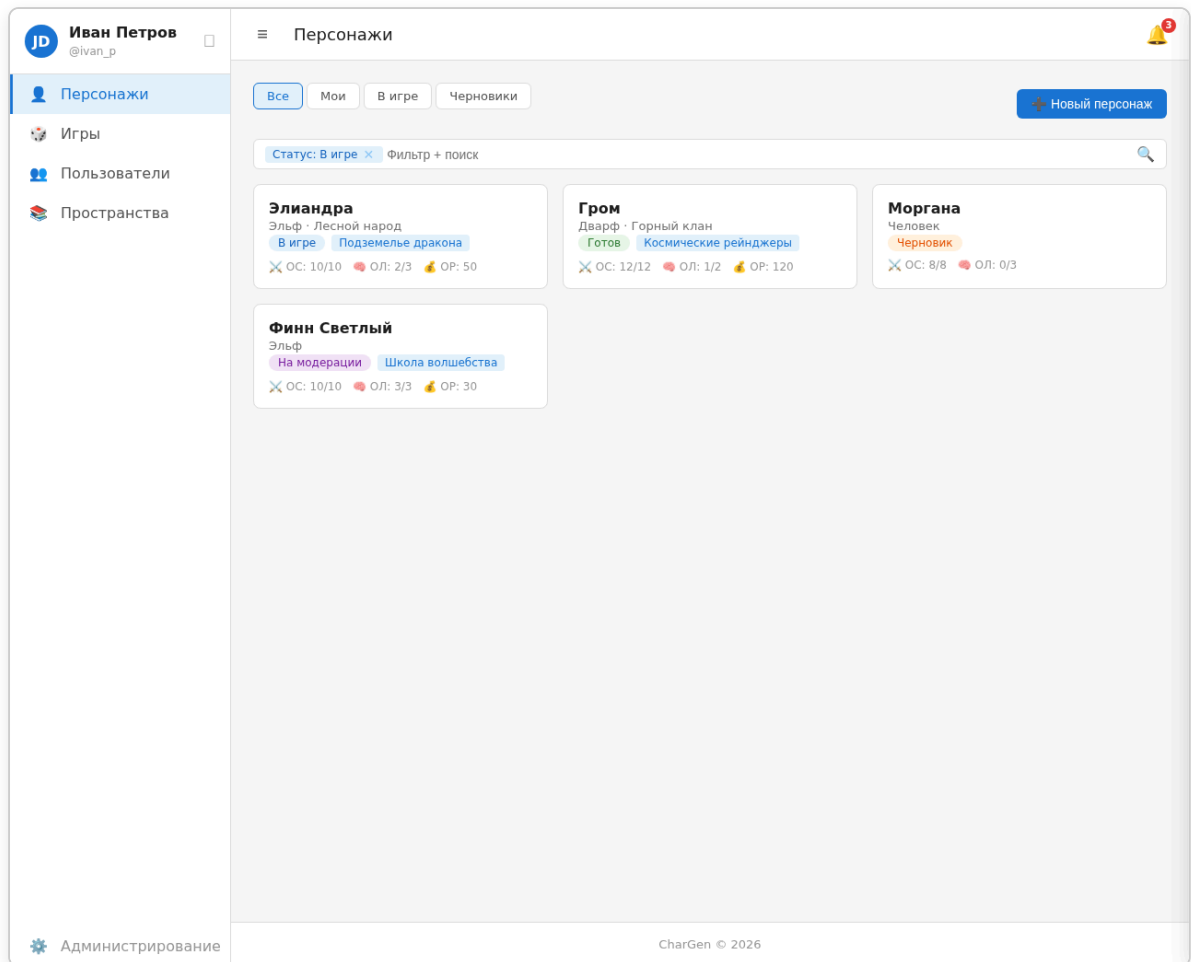


Страницы

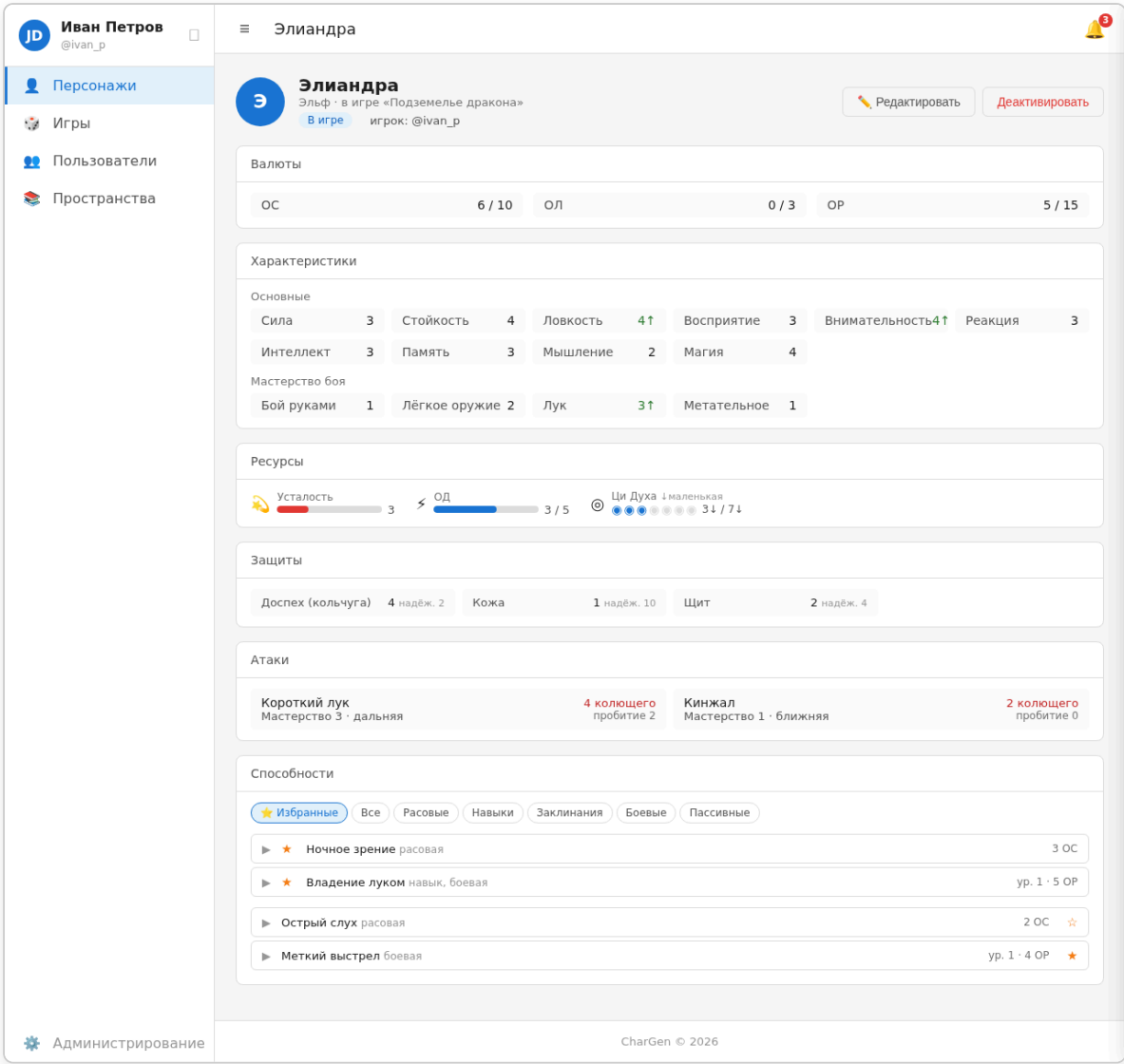
Путь	Страница	Описание	Доступ
<code>/games</code>	Список игр. Фильтр по статусу, названию. Карточки: название, статус, владелец, число участников.	Все	
<code>/games/new</code>	Создание игры. Название, краткое описание (для карточки в списке), полное описание, пространство правил, статус.	<code>game.create</code>	
<code>/games/:id</code>	Страница игры. Название, описание, статус, правила (пространство+время, лимиты ОС/ОР, теги/запретные теги), краткая сводка по персонажам.	По статусу игры	
<code>/games/:id/edit</code>	Основные настройки. Название, краткое описание (для списка), полное описание, статус	Владелец, ведущие	

Путь	Страница	Описание	Доступ
	(черновик/открытая/закрытая для просмотра/закрытая для вступления), картинка.		
<code>/games/:id/characters</code>	Персонажи игры. Полный список персонажей игры со статусами (в игре / вне игры / на модерации), фильтры, быстрый просмотр.	По статусу игры	
<code>/games/:id/rules</code>	Правила игры. Версия правил, лимиты ОС/ОР, теги, запретные теги.	Владелец, ведущие	
<code>/games/:id/members</code>	Участники. Список игроков, назначение ведущих, выдача индивидуальных прав.	Владелец, ведущие	
<code>/games/:id/invitations</code>	Приглашения. Отправленные, просмотренные, принятые, отклонённые.	Владелец, ведущие	
<code>/games/:id/moderate</code>	Модерация. Персонажи на модерации / требующие исправления.	Владелец, ведущие	
<code>/games/:id/loot</code>	Добыча. Список доступной/ добытой добычи, кнопки «Проявить интерес», раздача.	Владелец, ведущие	

Раздел «Персонажи» (`/characters`)



Макет: Карточка персонажа (/characters/:id)



Страницы

Путь	Страница	Доступ
/characters	Список персонажей. Фильтр: имя, раса, игра, статус, владелец, количество потраченной валюты (ОС/ОЛ/ОР).	Все (видят только доступных персонажей)
/characters	Список персонажей. Фильтр: имя, раса, игра, статус, владелец, кол-во потраченной валюты (ОС/ОЛ/ОР). Черновики видны только владельцу.	Все (видят только доступных)
/characters/new	Создание. Сначала выбор: свободное создание (ОС/ОР	character.create

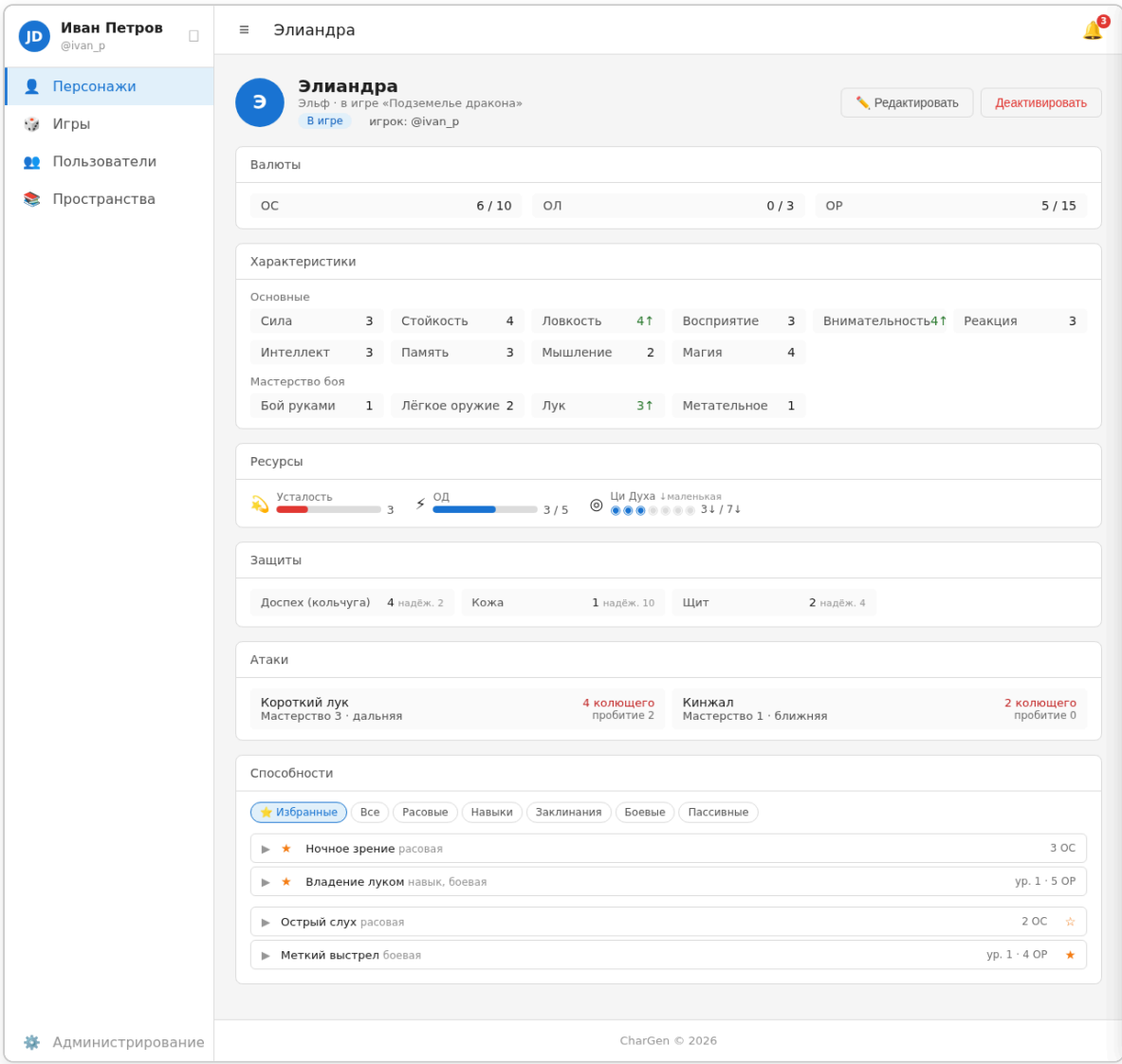
Путь	Страница	Доступ
	опционально, можно задать позже в редакторе) или через игру (ОС/ОР от игры, тоже можно изменить позже). ОЛ не задаётся — определяется возрастом и расой. После выбора → переход в редактор.	
<code>/characters/:id</code>	Карточка персонажа. Аватар, имя, раса, владелец, игра, статус, характеристики, ресурсы, способности, валюты, эффекты.	По правам
<code>/characters/:id/edit</code>	Редактирование. Одна страница с табами: Раса → Основа → Личность → Развитие → Закупка (+ будущий: История).	Владелец. В игре — сессионная модель (черновик → модерация)
<code>/characters/:id/versions</code>	История версий. Список версий, просмотр, откат.	Владелец
<code>/characters/:id/migrate</code>	Перевод на новую версию правил. Выбор версии, diff, запуск миграции → копия-черновик на новой версии.	Владелец
<code>/characters/:id/deactivate</code>	Деактивация. Персонаж скрывается из списков, данные сохраняются.	Владелец

Карточка персонажа (блоки)

- **Шапка:** аватар, имя, раса, игра, статус (черновик/готов/в игре/на модерации), игрок.
- **Валюты:** потрачено/осталось ОС, ОЛ, ОР.
- **Характеристики:**
 - **Основные:** Сила, Стойкость, Ловкость, Восприятие (Внимательность, Реакция), Интеллект (Память, Мышление), Магия.
 - **Мастерство боя:** динамический список (Бой руками, Лёгкое оружие, Лук, Метательное и т.д.) — то, что есть у персонажа.
- **Ресурсы:** динамический список, определяется правилами. Каждый ресурс: название, текущее значение, максимальное (опционально, для ресурсов без капа — только значение), размер (опционально, для размерных величин). Типы отображения: шкала (bar), числовое, кружки.

- **Защиты:** список с названием, значением защиты и надёжностью (например, «Доспех (кольчуга): 4, надёжность 2»).
- **Атаки:** оружие с мастерством, типом дистанции, уроном (число + тип), пробитием.
- **Способности:** grouped by type (расовые, навыки, заклинания, боевые, пассивные), фильтры, возможность отметить избранные (★) для вынесения в отдельный список.

Макет: Карточка персонажа (/characters/:id)



Раздел «Пользователи» (/users)

Путь	Страница	Доступ
/users	Список пользователей. Аватар, имя, фамилия, логин. Скрытые поля — только	user.view

Путь	Страница	Доступ
	с <code>user.view_sensitive</code> . Кнопка «+ Создать» — с <code>user.create</code> .	
<code>/users/new</code>	Создание пользователя.	<code>user.create</code>
<code>/users/:id</code>	Профиль. Аватар, имя, фамилия, логин, персонажи, игры. Права/группы — если есть доступ. Кнопка «Редактировать» — с <code>user.edit</code> .	<code>user.view</code>
<code>/users/:id/edit</code>	Редактирование пользователя. Аватар (drag-n-drop / выбор файла), логин, пароль, email, имя, фамилия, псевдоним. Группы — только с <code>user.edit</code> .	<code>user.edit</code>
<code>/users/:id/deactivate</code>	Деактивация пользователя. Отдельная страница: дата окончания (опционально), причина.	<code>user.deactivate</code>

Деактивация других сущностей: для игр (`/games/:id/deactivate`), пространств (`/spaces/:id/deactivate`), персонажей (`/characters/:id/deactivate`) — кнопка с попап-подтверждением, без даты и причины.

Иван Петров @ivan_p

Редактирование профиля

Персонажи

Игры

Пользователи

Пространства

Основная информация

Логин:

Email:

Имя:

Фамилия:

Псевдоним (никнейм):

Смена пароля

Текущий пароль:

Новый пароль:

Сохранить изменения

Группы

Игрок (изменение групп — только с правом 'user.edit')

Администрирование

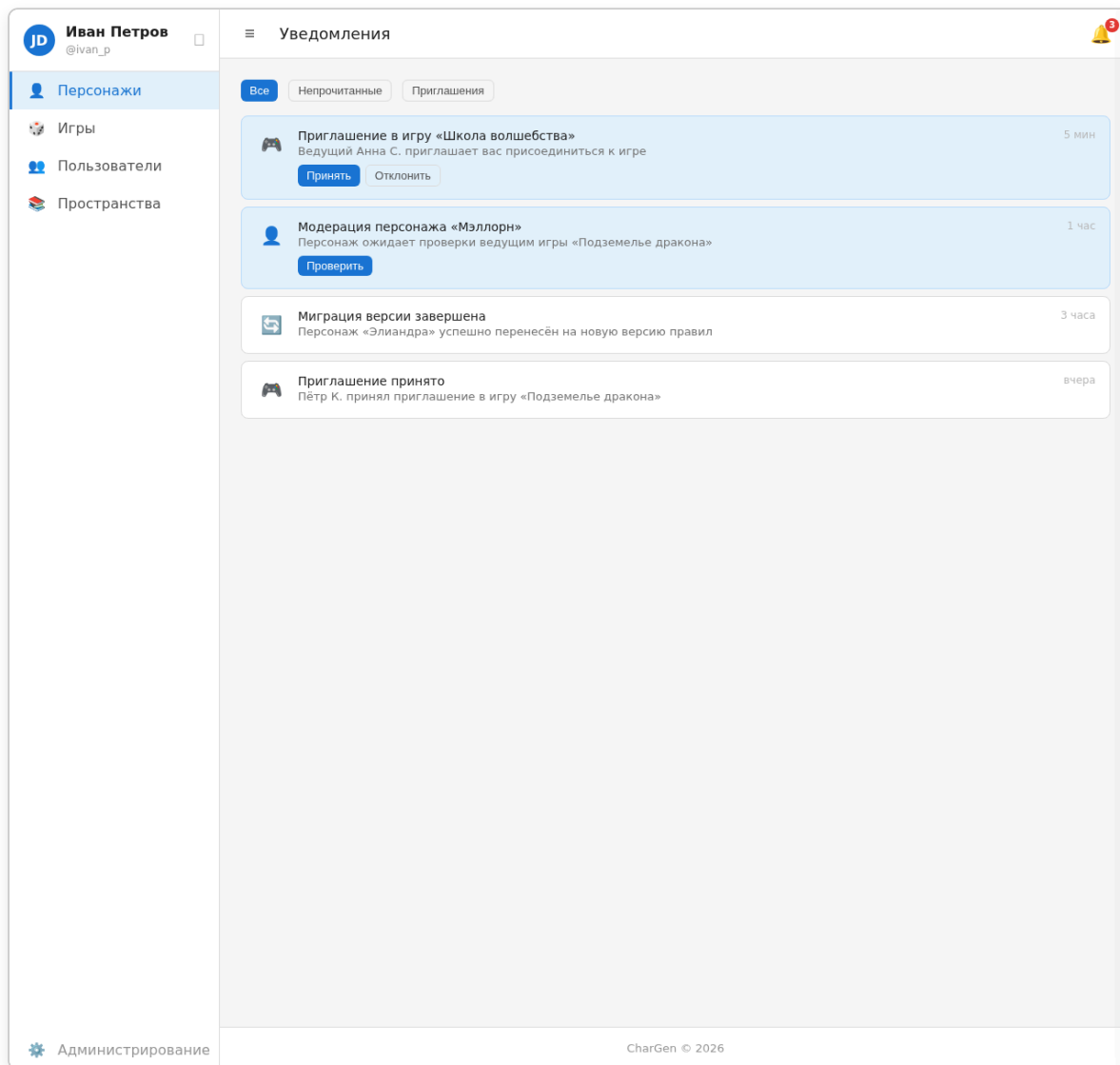
CharGen © 2026

Раздел «Уведомления»

- **Топбар:** иконка со счётчиком непрочитанных уведомлений. При клике — выезжает панель (slider) справа с последними уведомлениями, ссылка на полный список.
- **По клику:** открывается сайдпанель (Bitrix24-стиль) — тот же роут `/notifications` в overlay-режиме с крестиком.
- **Полная страница:** `/notifications` — группировка, сортировка, фильтрация, просмотр чужих уведомлений (для админов).
- **Администрирование:**
 - `/admin/notification-templates` — список шаблонов (`notification_template.view`)
 - `/admin/notification-templates/new` — создание (`notification_template.create`)

- `/admin/notification-templates/:id/edit` — редактирование (`notification_template.edit`)
- `/admin/notification-templates/:id/delete` — удаление (`notification_template.delete`)
- **Типы уведомлений:** приглашение в игру (кнопки Принять/Отклонить), модерация персонажа, миграция версии завершена.

Макет: Уведомления (/notifications)



Раздел «Группы» (`/admin/groups`)

Путь	Страница	Доступ
<code>/admin/groups</code>	Список групп. Название, число участников, статус.	<code>group.view</code>

Путь	Страница	Доступ
<code>/admin/groups/new</code>	Создание группы. Название, описание, назначение прав (группированные чекбоксы).	<code>group.create</code>
<code>/admin/groups/:id</code>	Карточка группы. Участники, права.	<code>group.view</code>
<code>/admin/groups/:id/edit</code>	Редактирование группы. Изменение названия, участников, прав.	<code>group.edit</code>
<code>/admin/groups/:id/deactivate</code>	Деактивация группы.	<code>group.deactivate</code>

Раздел «Теги» (`/admin/tags`)

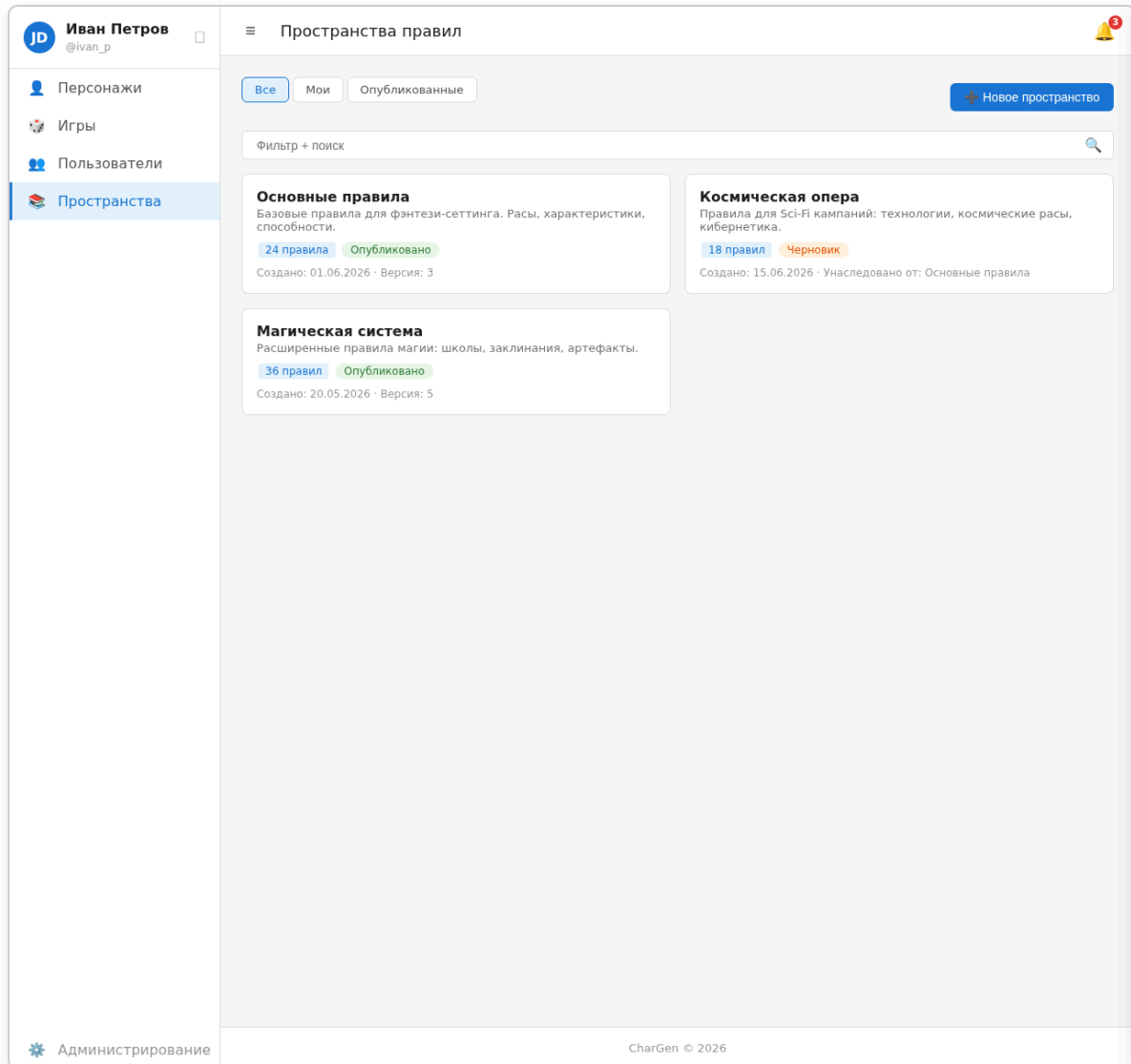
Путь	Страница	Доступ
<code>/admin/tags</code>	Список тегов. <code>string_id</code> , <code>name</code> , <code>description</code> , флаг активности.	<code>tag.view</code>
<code>/admin/tags/new</code>	Создание тега. <code>string_id</code> (уникальный, латиница/цифры/подчёркивание), <code>name</code> (отображаемое имя), <code>description</code> (опционально).	<code>tag.create</code>
<code>/admin/tags/:id/edit</code>	Редактирование тега.	<code>tag.edit</code>
<code>/admin/tags/:id/delete</code>	Удаление тега (soft-delete).	<code>tag.delete</code>

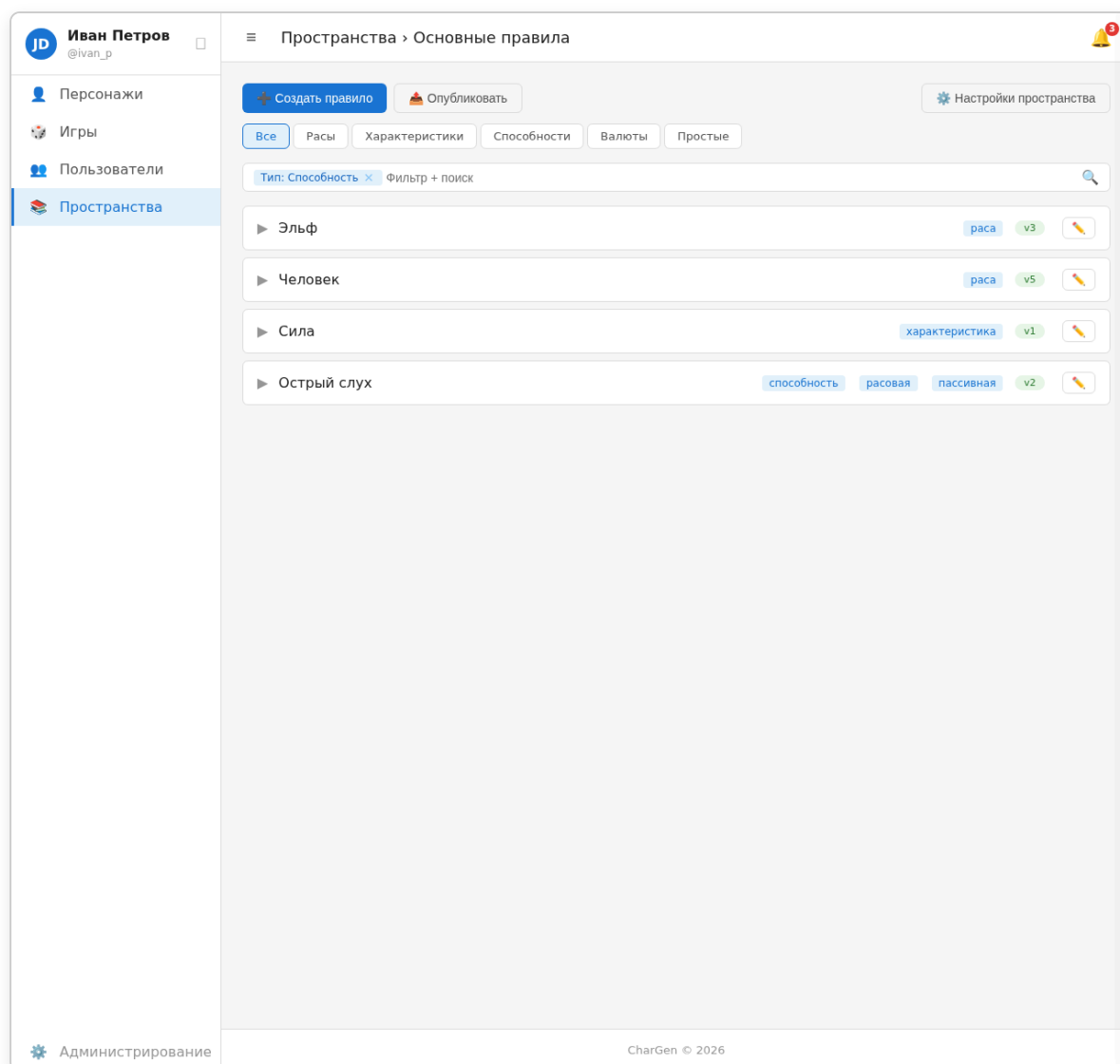
Добавление тегов к правилу: во время редактирования правила — выпадающий список / combobox с поиском по `name`. Справа кнопка «+» — открывает попап создания нового тега (`string_id` генерируется из `name` или вводится вручную). После сохранения попапа тег сразу доступен для выбора без перезагрузки.

Раздел «Пространства» (`/spaces`)

Путь	Страница	Доступ
<code>/spaces</code>	Список пространств. Название, дата создания, статус.	<code>space.view_all</code>

Путь	Страница	Доступ
<code>/spaces/new</code>	Создание. Название, описание, опционально «наследовать от» (выбор родительского пространства — одноразово, копирование снепшота).	<code>space.create</code>
<code>/spaces/:id</code>	Страница пространства. Дерево/список правил на текущий момент. Селектор временного среза. Кнопки: создать правило, опубликовать, настройки.	<code>space.view</code>
<code>/spaces/:id/rules/new</code>	Создание правила. Выбор типа, форма.	<code>rule.create</code>
<code>/spaces/:id/rules/:ruleId</code>	Просмотр правила. Текущее состояние, история версий, diff между версиями.	<code>space.view</code>
<code>/spaces/:id/rules/:ruleId/edit</code>	Редактирование. Создание новой версии.	<code>rule.edit</code>
<code>/spaces/:id/publish</code>	Публикация. Выбор целевого пространства, выбор набора правил, diff, подтверждение.	<code>space.edit</code>
<code>/spaces/:id/settings</code>	Настройки. Название, описание, права доступа (per-space: группы + индивидуально).	<code>space.edit</code>
<code>/spaces/:id/deactivate</code>	Деактивация пространства.	<code>space.edit</code>





Редактор правил

Для каждого типа правила — своя форма. Общие части:

- **Общие поля:** UUID (авто), строковой ID (опционально, для URL), название, описание (HTML, WYSIWYG), теги.
- **Спецификация:** JSON-блоки структурированных данных (требования, стоимости, grants, level_scaling и т.д.).

Типоспецифичные поля:

Тип	Поля
Простое правило	Только общие
Раса	<code>parent_race_id</code> , авторасчёт ОС (вычисляет стоимость расы, а не лимит персонажа), превью наследования
Характеристика	<code>is_dimensional</code> , <code>min_value</code> , <code>max_value</code> , <code>group</code> , <code>formula</code>

Тип	Поля
Валюта	<code>key</code> (системное имя), <code>name</code> (основное имя)
Способность (черты, навыки, действия, процессы)	<code>cost[]</code> , <code>hard</code> (трудность), <code>levels</code> , <code>parent_ability_id</code> , <code>automatic</code> (флаг), <code>visibility[]</code> (где показывать: <code>основа</code> , <code>личность</code> , <code>развитие</code> , <code>игра</code> , <code>всегда</code>). Спецификация: <code>requirements</code> , <code>grants</code> , <code>action_costs</code> , <code>level_scaling</code>
Заклинание	Наследует все поля способности + спецификацию (см. «Заклинания» ниже)
Предмет	<code>category</code> (money/equipment/other), <code>consumable</code> , <code>weight</code> , <code>cost_gm</code> , <code>durability</code> . В спецификации: <code>attacks[]</code> (для оружия), <code>defense_slots[]</code> (для брони), <code>block</code> (для щита), <code>subtypes[]</code> (для equipment).
Эффект	🟡 Отложен

Режим редактирования: при создании/редактировании персонажа пользователь с правом `rule.edit` может включить **«Режим редактирования»** — интерфейс позволяет править правила пространства прямо в контексте создания персонажа, без перехода на страницу пространства.

Общий лейаут

Топбар (`v-app-bar`)

- Слева: кнопка  — скрыть/показать сайдбар
- Центр: название текущей страницы (раздела)
- Справа:  иконка уведомлений со счётчиком (badge). При клике — выезжает панель (slider) справа с последними уведомлениями. Полная история — на `/notifications`.

Сайдбар (`v-navigation-drawer` , `collapsible`)

- **Блок пользователя** (вверху): аватар, имя, @логин
- **Пункты меню** (иконка + текст): Персонажи, Игры, Пользователи, Пространства
- Внизу (при наличии прав): **Администрирование** (`/admin/groups`)

Футер (`v-footer`)

- Копирайт, ссылки

Контекст пространства-времени

- В топбаре не отображается
- Показывается внутри страницы, где это уместно (редактор правил, создание персонажа для игры и т.д.) — селектор пространства и/или даты версии

Bitrix-style фильтр

Используется на всех страницах со списками (игры, правила, расы, черты):

- **filter-bar** — кликабельное поле с бордером. Содержит:
 - Чипсы активных фильтров слева
 - Текстовый ввод справа от чипсов (placeholder «Фильтр + поиск»)
 - Иконка 🔍 справа
- **filter-popup** — выпадающий попап, открывается по клику/фокусу на filter-bar:
 - Поля фильтрации
 - Тумблеры (**filter-toggle**) включения/отключения каждого поля
 - Кнопки «Применить» / «Сбросить»
- ✕ на чипсе — удаляет фильтр без открытия попапа

Horizontal editor panel

Панель под топбаром на странице редактора персонажа:

- Flex-строка с блоками-этапами: каждый блок — кликабельный, переключает содержимое ниже
- **Блоки** (слева направо): Раса (показывает стоимость расы), Врождённые черты (OC spent/total), Личность (OL spent/total), Развитие (OP spent/total), Инвентарь
- Активный этап подсвечен синим фоном
- Справа — **чипсы основных характеристик** (сетка 2×3, полные названия):
 - Показываются только основные характеристики (базовые для расы)
 - Только ненулевые
 - Если раса не выбрана (характеристик нет) — блок скрыт целиком

Expansion panel

Раскрывающийся список для рас, черт, способностей и любых существ с краткой и полной информацией:

- **Header** (кликабельный, открывает/закрывает тело):
 - Стрелка ▶ (поворачивается при открытии)
 - Краткая информация (название, стоимость, теги)

- **Кнопка**  — в крайней правой части шапки. Отображается только при наличии права `rule.edit`. Открывает редактирование правила в попапе или сайдпанели.
- **Счётчик +/-** (только для черт/способностей с уровнями)
- **Body** (скрыт по умолчанию):
 - Полное описание правила
 - Требования и условия
 - Теги
 - Дополнительная информация (характеристики, стоимость по уровням и т.д.)
- Состояния:
 - `open` — тело видимо, стрелка повернута вниз
 - `selected` — подсветка синей границей и фоном (выбранный элемент). Выбранные элементы отображаются в начале списка.

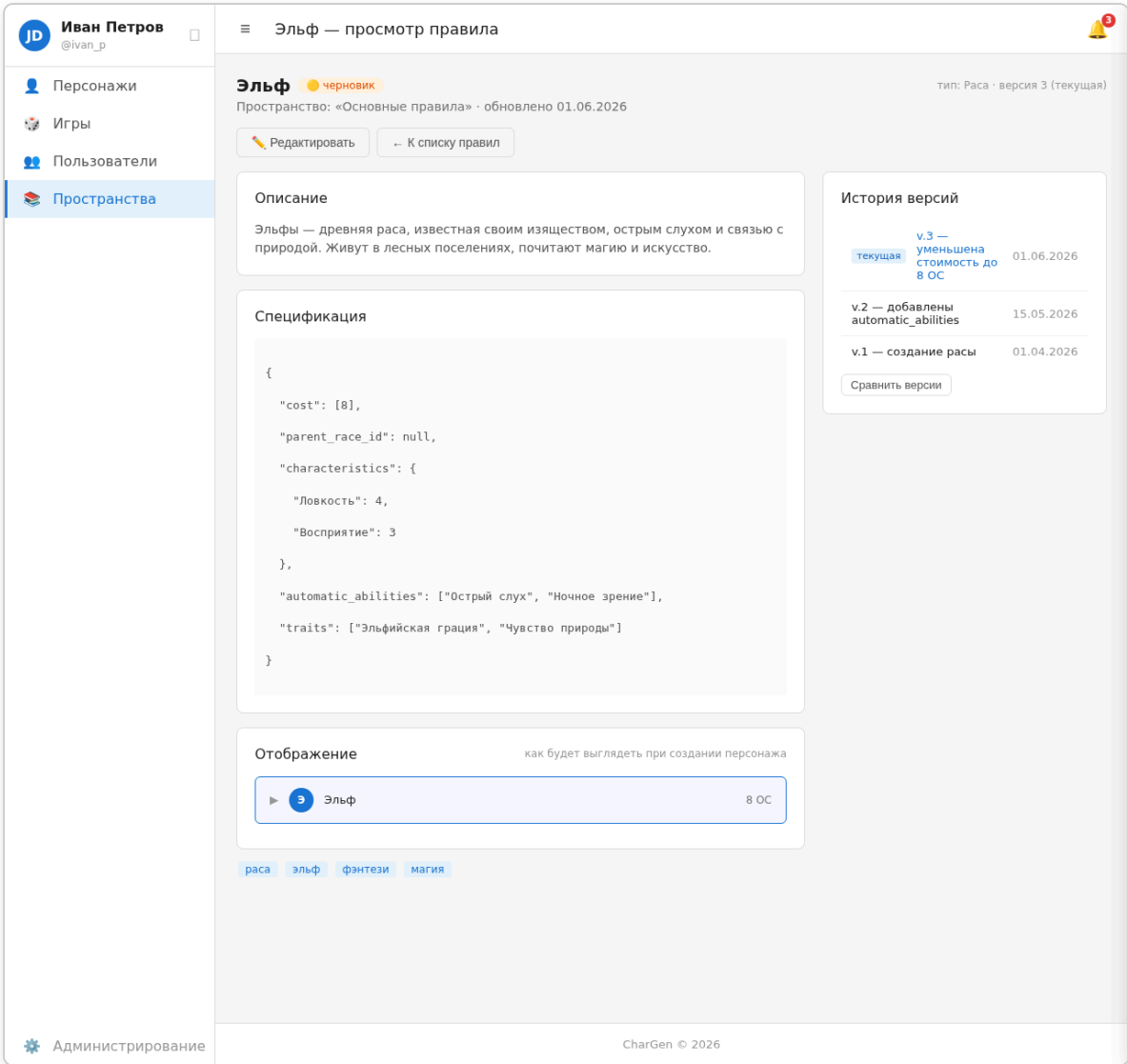
Сводка решений Этапа 7

1. **✓ Раздел «Игры»:** список, создание, страница игры, основные настройки, персонажи, правила, участники, приглашения, модерация, добыча — отдельные подстраницы.
2. **✓ Статусы игры:** Черновик, Открытая, Закрытая для просмотра, Закрытая для вступления.
3. **✓ Раздел «Персонажи»:** список (фильтры: имя, раса, валюта), создание (свободное/через игру), карточка, редактор (табы: раса → ОС → ОЛ → ОР+будущие), версии, миграция версий правил, деактивация.
4. **✓ Боевая карточка** — отдельная страница, реализация после основных разделов.
5. **✓ Редактор персонажа** — одна страница с табами, не wizard.
6. **✓ Удаление персонажа** — деактивация (soft-delete).
7. **✓ Черновики** в списке персонажей видны только владельцу.
8. **✓ Раздел «Пользователи»:** 5 страниц (`/users` , `/users/new` , `/users/:id` , `/users/:id/edit` , `/users/:id/deactivate`).
9. **✓ Раздел «Группы»:** 5 страниц (`/admin/groups/*`).
10. **✓ Раздел «Пространства»:** 9 страниц (`/spaces` , `/spaces/new` , `/spaces/:id` , `/spaces/:id/rules/new` , `/spaces/:id/rules/:ruleId` , `/spaces/:id/rules/:ruleId/edit` , `/spaces/:id/publish` , `/spaces/:id/settings` , `/spaces/:id/deactivate`).
11. **✓ Редактор правил:** отдельные формы для каждого типа (Простое, Раса, Характеристика, Валюта, Способность, Заклинание, Предмет, Эффект — отложен). Формы: общие поля + HTML-описание + спецификация (JSON-блоки).

12.  **Режим редактирования:** при создании персонажа с `rule.edit` можно править правила в контексте.
13.  **Уведомления:**  в топбаре (slider с последними), `/notifications` (полная история). `/admin/notification-templates` (список), `/admin/notification-templates/new`, `/admin/notification-templates/:id/edit`.
14.  **Сброс пароля:** стандартный (письмо со ссылкой).
15.  **Теги:** `/admin/tags` (список), `/admin/tags/new`, `/admin/tags/:id/edit`.
Добавление тегов к правилу — через combobox + попап создания нового тега.
16.  **Аватар — загрузка в профиле** (`/users/:id/edit`, drag-n-drop / выбор файла).
17.  `/characters/new` — проверка права `character.create`.
18.  **Деактивация:** для пользователей — отдельная страница (дата, причина).
Для игр/пространств/персонажей — кнопка + попап (без даты и причины).
19.  **Общий лейаут:** топбар (≡ + название +  уведомления со slider), сайдбар (collapsible, блок пользователя + аватар + меню + администрирование), футер.
20.  **Контекст пространства-времени** — внутри страницы, не в топбаре.

Заклинания

Заклинания — подтип Способности, наследуют все её поля (`cost[]`, `hard`, `levels`, `parent_ability_id`, `automatic`, `visibility[]`, требования, `grants`, `action_costs`, `level_scaling`).



Спецификация заклинания (JSON-блоки в **списке**)

```
{  "type": "spell",  "difficulty": { "value": 2, "size": null },  "duration": { "type": "sustained", "difficulty": { "value": 2, "size": null } },  "components": { "somatic": true, "verbal": true, "material": "Горсть пепла" },  "target": { "type": "creature", "count": 1, "range": { "value": 10, "unit": "m" } }}
```

Поле	Тип	Описание
difficulty	размерное число {value, size}	Сложность сотворения заклинания. В будущем может разделиться на два поля — Контроль и Мощность. Пока одна сложность.
duration	объект	Длительность (см. ниже)

Поле	Тип	Описание
<code>components</code>	<code>{somatic: bool, verbal: bool, material: string null}</code>	Компоненты
<code>target</code>	<code>{type, count, range}</code>	Цель (опционально)

Типы длительности

<code>type</code>	Поля	Описание
<code>instant</code>	—	Мгновенное, длительности нет
<code>sustained</code>	<code>difficulty: размерное число</code>	Сложность поддержания (та же, что и при сотворении). Опционально <code>time_limit</code>
<code>refreshable</code>	<code>difficulty: размерное число</code> , <code>action_cost: \{value, unit: "od"\}</code>	Сложность обновления и время на обновление. Опционально <code>time_limit</code>
<code>timed</code>	<code>time_limit: \{value, unit: "turn" "minute" "hour"\}</code>	Предельное время работы

`time_limit` — опциональное поле для `sustained` и `refreshable` (предел, сколько ходов/минут можно поддерживать или обновлять).

Визуализация

- В карточке персонажа — в том же expansion panel, что способности, с тегом `заклинание`
- Фильтр по тегу `заклинание` скрывает/показывает только заклинания

Предметы

Макет: Карточка персонажа (/characters/:id)

JD

Иван Петров

@ivan_p

Персонажи

Игры

Пользователи

Пространства

Элиандра

Э

Элиандра

Эльф · в игре «Подземелье дракона»

В игре · игрок: @ivan_p

Редактировать

Деактивировать

Валюты

ОС

6 / 10

ОЛ

0 / 3

ОР

5 / 15

Характеристики

Основные

Сила

3

Стойкость

4

Ловкость

4↑

Восприятие

3

Внимательность

4↑

Реакция

3

Интеллект

3

Память

3

Мышление

2

Магия

4

Мастерство боя

Бой руками

1

Лёгкое оружие

2

Лук

3↑

Метательное

1

Ресурсы

Усталость

3

ОД

3 / 5

Ци Духа ↓ маленькая

3↓ / 7↓

Защиты

Доспех (кольчуга)

4 надёж. 2

Кожа

1 надёж. 10

Щит

2 надёж. 4

Атаки

Короткий лук

Мастерство 3 · дальняя

4 колющего пробитие 2

Кинжал

Мастерство 1 · ближняя

2 колющего пробитие 0

Способности

Избранные

Все

Расовые

Навыки

Заклинания

Боевые

Пассивные

Ночное зрение

расовая

3 ОС

Владение луком

навык, боевая

ур. 1 · 5 ОР

Острый слух

расовая

2 ОС

☆

Меткий выстрел

боевая

ур. 1 · 4 ОР

☆

Администрирование

CharGen © 2026

Спецификация предмета (JSON-блоки в **спис**)

```
{
  "blocks": [
    {
      "type": "item_base",
      "category": "equipment",
      "subtypes": ["weapon", "shield"],
      "consumable": false,
      "weight": { "value": 3, "size": 0 },
      "cost_gm": 500,
      "durability": 10
    },
    {
      "type": "item_attack",
      "name": "Рубящий",
      "range": "melee",
      "damage_type": "slashing",
      "damage": { "type": "formula", "value": "@{strength}+2" },
      "penetration": { "type": "static", "value": 1 },
      "accuracy": 0
    }
  ]
}
```

```

    },
    {
      "type": "item_block",
      "efficiency": { "value": 1, "size": 0 },
      "defense": { "value": 2, "size": 0 }
    }
  ]
}

```

Категории и подтипы

Категория	Подтипы	Описание
money	—	Монеты. Стоимость в gm, отображение в формате «5 гз, 7 гс, 9 гм»
equipment	weapon, armor, shield (массив, может быть несколько)	Снаряжение. Можно экипировать.
other	—	Прочее (ключи, документы, ингредиенты)

Урон и пробитие (DamageFormula)

```

{ "type": "static", "value": 3 }
{ "type": "formula", "value": "@{strength}+2" }

```

Где `@{strength}` — ссылка на характеристику персонажа.

Инвентарь

`character_inventory(character_version_id, item_rule_id, quantity, durability_left, equipped BOOL)`

- Инвентарь привязан к конкретной версии персонажа.
- При `copy-on-write` инвентарь копируется вместе с остальными данными.

Добыча игры

`game_loot(id, game_id, item_rule_id, quantity, notes, status, created_at)`

Статус	Описание
available	Возможная добыча. Не принадлежит никому.
acquired	Добыта, но ещё не роздана
distributed	Роздана персонажам

Механика:

- Игроки нажимают «Проявить интерес» → запись в `game_loot_interest(loot_id, user_id)`
- GM видит, кто заинтересован, выбирает получателя → предмет перемещается в `character_inventory`, loot-запись помечается `distributed`
- Раздача добычи — действие GM, не требует модерации

Чат

Типы чатов

Тип	Описание
<code>private</code>	Личный чат 1-на-1 между двумя пользователями
<code>group</code>	Групповой чат (произвольный набор участников)
<code>game</code>	Чат игры (все участники игры автоматически в нём)

Новые типы добавляются без миграции схемы.

Сообщения

- Хранятся в `chat_messages(id, chat_id, user_id, content, dice_result, created_at)`
- Подгрузка истории (scroll up)
- Поддержка встроенных результатов бросков

Команда броска кубиков

```
/roll Nd6[:efficiency] [adv:N] [dis:N] [label:текст]
```

Примеры:

- `/roll 5d6 e:3` — 5 кубов, эффективность 3
- `/roll 5d6 e:3 adv:1` — 5 кубов + 1 преимущество → всего 6, убрать 1 худший результат
- `/roll 5d6 e:3 dis:2` — 5 кубов + 2 помехи → всего 7, убрать 2 лучших результата

Подсчёт успехов для каждого куба d6:

- `1` → 2 успеха

- `[2..efficiency]` → 1 успех
- `[efficiency+1..5]` → 0 успехов
- `6` → -1 успех

Преимущества/помехи:

- Преимущество (adv): добавить N кубов к броску, перед подсчётом убрать N худших результатов
- Помеха (dis): добавить N кубов к броску, перед подсчётом убрать N лучших результатов

Размер успехов = размер проверяемой характеристики.

Пример вывода:

```
Проверка на Силу. Сила 5↑. Эффективность 3.
Бросок: 1, 1, 2, 5, 6
1 → 2 успеха
1 → 2 успеха
2 → 1 успех (2 ≤ 3)
5 → провал
6 → -1 успех

Итого: 4 успеха.
Сила большая → 4↑ успеха.
```

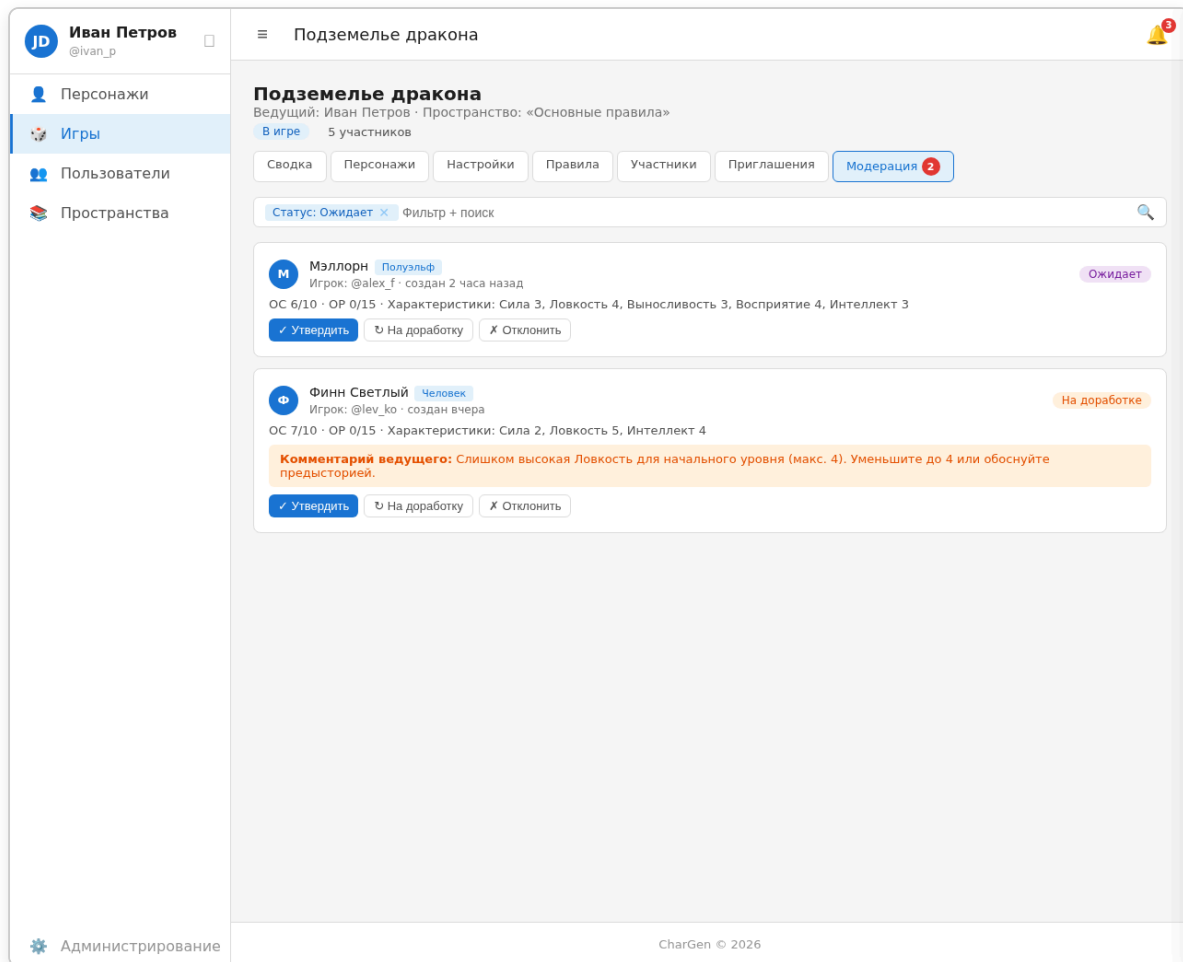
Макросы

Хранятся в `user_macros(id, user_id, name, text_template, roll_formula, efficiency, created_at)`.

- Пользователь создаёт макрос в своём профиле
- Макрос отображается как кнопка в интерфейсе чата
- По нажатию: в чат отправляется сообщение вида `"text_template. Бросок Nd6 = ... Итого: X успехов."`
- Пример: `name: "Удар", text: "Атака мечом", roll: "3d6", efficiency: 3`

Модерация добычи

- **Страница добычи игры** — список `available` / `acquired` предметов, фильтр по статусу
- **Кнопка «Проявить интерес»** — на карточке предмета в списке добычи
- **Раздача** — попап выбора получателя (список игроков), подтверждение → предмет уходит в инвентарь



Макеты: что ещё нужно сделать

Список макетов, которые стоит реализовать визуально (уникальный UI, сложно описать текстом):

№	Путь	Страница	Причина
1	/games/new	Создание игры	Выбор пространства, выставление лимитов ОС/ОП/ОЛ, теги
2	/games/:id/moderate	Модерация персонажей	Список на утверждение/отклонение с комментарием, кнопки действий
3	/characters/new	Выбор способа создания	Переключатель: свободное создание vs через игру. Уникальный UI
4	/characters/:id	Карточка персонажа	Read-only просмотр всех блоков (характеристики,

№	Путь	Страница	Причина
			ресурсы, способности, валюты)
5	<code>/spaces/new</code>	Создание пространства	Выбор наследования (родительское пространство), копирование снимка
6	<code>/spaces/:id/rules/:ruleId</code>	Просмотр правила	История версий, diff между версиями
7	<code>/spaces/:id/publish</code>	Публикация пространства	Выбор набора правил, просмотр diff, подтверждение
8	<code>/spaces/:id/settings</code>	Настройки пространства	Per-space права доступа (группы + индивидуальные)
9	<code>/characters/:id/edit?step=inventory</code>	Закупка предметов	Список предметов с ценами, бюджет, корзина
10	<code>/games/:id/loot</code>	Добыча игры	Список добычи, кнопки «Проявить интерес», раздача
11	<code>/chat/:id</code>	Чат	Лента сообщений, поле ввода, кнопки макросов

Не требуют отдельных макетов (описываются текстом или повторяют существующие):

- `/reset-password/:token` — минимальная форма (2 поля), как `/login`
- `/games/:id/edit` — похожа на `/games/new`
- `/games/:id/rules` — как `space-rules.html`
- `/games/:id/members`, `/games/:id/invitations` — таблица со статусами
- `/users`, `/users/new`, `/users/:id`, `/users/:id/edit` — таблица и формы, вторичны по UI
- Деактивация (все сущности) — попап-подтверждение
- `/characters/:id/versions`, `/characters/:id/migrate`, `/characters/:id/deactivate`
- `/spaces/:id/rules/new`, `/spaces/:id/rules/:ruleId/edit`
- `/spaces/:id/deactivate`
- `/admin/groups/*`, `/admin/tags/*`, `/admin/notification-templates/*`
- `/profile/macros` — форма списка/создания макросов

UI-паттерны, требующие описания

Следующие интерфейсы есть в ТЗ на уровне функциональности, но не имеют проработанного UI-описания:

1. **Diff версий (правил и персонажей)** — как визуально показывать изменения в характеристиках, способностях, валютах, JSON-спецификации. Подсветка добавленного/удалённого/изменённого.
2. **Модерация персонажа** — UI утверждения/отклонения с полем для комментария ведущего, уведомление владельца. ✓ Макет: `game-moderate.html`.
3. **Приглашения**

Страница `/games/:id/invitations`:

- Таблица со строками: аватар + имя/логин приглашённого, роль (игрок/ведущий), статус, дата отправки, дата ответа, действия (отозвать).
- Статусы: `отправлено`, `просмотрено` (уведомление прочитано), `принято`, `отклонено`.
- Фильтры (Bitrix-style): по статусу, по роли, по имени/логину.
- Кнопка «+ Пригласить» — открывает попап.

Попап отправки приглашения:

- Поле ввода: логин или email (с autocomplete по пользователям системы).
- Селектор роли: Игрок / Ведущий.
- Кнопка «Отправить».
- При отправке: создаётся уведомление получателю (иконка 🗨, заголовок «Приглашение в игру «Название»», кнопки «Принять» / «Отклонить» в теле уведомления).

Поведение:

- Принятое приглашение: пользователь добавляется в участники игры, строка в таблице становится неактивной (серый фон), появляется дата ответа.
- Отклонённое: строка становится неактивной, статус красно-серый, actions скрыты.
- Отозванное (ведущий нажал «Отозвать»): статус меняется на `отозвано`, уведомление получателя аннулируется (если не прочитано) или помечается как неактуальное.

4. **Публикация пространства** — UI выбора правил для публикации (чекбоксы/переключатели), просмотр diff с целевым пространством, подтверждение. ✓ Макет: `space-publish.html`.
5. **Настройки пространства** — UI per-space прав доступа: выбор групп, индивидуальные пользователи, матрица прав. ✓ Макет: `space-settings.html`.
6. **Страница версий персонажа** — UI списка версий (дата, автор, комментарий), просмотр конкретной версии, diff, кнопка отката.

7. **Миграция версий правил** (`/characters/:id/migrate`) — UI выбора целевой версии, diff изменений, запуск миграции (создание черновика).
8. **Редактор правил** — UI типоспецифичных форм: как выглядят поля для каждого типа правила, как редактировать JSON-блоки спецификации.
9. **Режим редактирования** — UI переключения в контексте создания/редактирования персонажа, визуальное отличие фона/рамок editable-элементов.
10. **Создание персонажа** — UI выбора свободного создания vs игры с предпросмотром лимитов.  Макет: `character-new.html` .
11. **Закупка предметов** (`/characters/:id/edit?step=inventory`) — список доступных предметов с ценами, бюджет (валюта «Деньги»), корзина выбранного, кнопки +/- количества. Остаток → монеты.
12. **Добыча игры** (`/games/:id/loot`) — таблица/карточки предметов со статусом, кнопка «Проявить интерес» для игроков, попап раздачи для GM (выбор получателя → подтверждение).
13. **Чат** (`/chat/:id`) — лента сообщений с подгрузкой истории, поле ввода с поддержкой `/roll` , inline-отображение результатов броска. Кнопки макросов рядом с полем ввода.
14. **Макросы** (`/profile/macros`) — таблица макросов, форма создания (название, текст, roll-формула, эффективность). Список + попап создания.

Требуется дальнейшей детализации

Следующие пункты описаны на уровне концепции, но требуют дальнейшей проработки (без привязки к UI):

1. **Журнал действий (админ-лог)** — хранение действий пользователей (создание/изменение/удаление сущностей). Требуется: модель данных, какие события логировать, UI просмотра.
 2. **Боевая карточка** — отдельная страница, компактный вид для боя: действия, ресурсы, эффекты. Реализация после основных разделов.
 3. **Эффекты (runtime-статусы)** — тип правила с описанием: наложение, длительность, снятие. Требуется детальной проработки (см. Этап 2 — отложено).
 4. **Раздел «Правила» (документация)** — полный каталог с поиском и фильтрами. Не требует архитектурной подготовки, добавится позже.
-

Этап 8: Схема базы данных

Соглашения:

- `id` — автоинкремент (SERIAL/BIGINT UNSIGNED AUTO_INCREMENT), если не указано иное.
- `created_at` , `updated_at` — TIMESTAMP.
- `BOOL` — TINYINT(1).
- `→` — внешний ключ.
- Индексы перечислены под каждой таблицей.

Файлы

```
files(  
  id, owner_id → users.id NOT NULL,  
  filename VARCHAR,           -- имя на диске  
  original_name VARCHAR,      -- оригинальное имя при загрузке  
  mime VARCHAR,  
  size INT,  
  path VARCHAR,  
  created_at  
)  
INDEX: (owner_id)
```

Аутентификация

```
sessions(  
  id, user_id → users.id NOT NULL,  
  token VARCHAR UNIQUE,       -- хеш refresh-токена  
  expires_at TIMESTAMP NOT NULL,  
  created_at  
)  
INDEX: (user_id), (token)  
  
password_reset_tokens(  
  id, user_id → users.id NOT NULL,  
  token VARCHAR UNIQUE,       -- одноразовый токен  
  expires_at TIMESTAMP NOT NULL,  
  used BOOL DEFAULT false,  
  created_at  
)  
INDEX: (token), (user_id)
```

Макет: Вход в систему (/login)

CharGen

Войдите в систему управления персонажами

Логин или email

Пароль

☐ Запомнить меня

Войти

[Забыли пароль?](#) · [Регистрация](#)

Макет: Регистрация (/register)

Регистрация

Создайте учётную запись

Логин *

my_nickname

Email

mail@example.com

Рекомендуется для сброса пароля. Если заполнен — должен быть уникальным.

Пароль *

Подтверждение пароля *

Зарегистрироваться

Уже есть аккаунт? [Войти](#)

Сброс пароля

Введите логин или email. Если email привязан к аккаунту — на него придёт ссылка для сброса.

Логин или email

Отправить ссылку

[Вернуться ко входу](#)

Пользователи, группы, права

```
users(
  id, login VARCHAR UNIQUE NOT NULL,
  password_hash VARCHAR NOT NULL,
  email VARCHAR UNIQUE,          -- для сброса пароля, может быть пустым
  first_name VARCHAR, last_name VARCHAR, nickname VARCHAR,
  avatar_file_id → files.id NULL,
  super_admin BOOL DEFAULT false NOT NULL,
  active BOOL DEFAULT true NOT NULL,
  deactivated_until DATE NULL, deactivate_reason TEXT NULL,
  created_at, updated_at
)
INDEX: (email), (login)

groups(
  id, name VARCHAR NOT NULL,
  active BOOL DEFAULT true NOT NULL,
  created_at
)

user_groups(
  user_id → users.id NOT NULL,
  group_id → groups.id NOT NULL,
  protected BOOL DEFAULT false NOT NULL, -- true для супер-админа в группе Адм
  PRIMARY KEY (user_id, group_id)
)

group_permissions(                -- глобальные права
```

```

    group_id → groups.id NOT NULL,
    permission_key VARCHAR NOT NULL,          -- user.* | group.* | game.* | space
    PRIMARY KEY (group_id, permission_key)
)

```

Пространства и правила

```

spaces(
    id, name VARCHAR NOT NULL, description TEXT,
    owner_id → users.id NOT NULL,
    inherit_from → spaces.id NULL,          -- родитель при наследовании (только при со
    active BOOL DEFAULT true NOT NULL,
    created_at
)
INDEX: (owner_id)

space_permissions(                                -- per-space права
    space_id → spaces.id NOT NULL,
    assignee_type VARCHAR NOT NULL,            -- 'group' | 'user'
    assignee_id INT NOT NULL,
    permission_key VARCHAR NOT NULL,          -- space.view | space.comment | spac
    UNIQUE (space_id, assignee_type, assignee_id, permission_key)
)
INDEX: (space_id)

tags(
    id, string_id VARCHAR UNIQUE NOT NULL,
    name VARCHAR NOT NULL, description TEXT,
    active BOOL DEFAULT true NOT NULL
)

rules(                                            -- реестр правил (глобальный ID)
    rule_id UUID PRIMARY KEY,
    type VARCHAR NOT NULL                      -- simple | race | characteristic |
)

rule_versions(                                  -- версионирование правил
    id, rule_id → rules.rule_id NOT NULL,
    space_id → spaces.id NOT NULL,
    created_at TIMESTAMP NOT NULL,            -- (rule_id, space_id, created_at) U
    name VARCHAR NOT NULL, description_html TEXT, spec_json JSON,
    active BOOL DEFAULT true NOT NULL         -- false = правило удалено (маркер-в
)
INDEX: (space_id, rule_id, created_at DESC),    -- основной запрос «на момент T»
      (rule_id),                               -- все версии одного правила
      (space_id)                               -- все версии в пространстве

rule_tags(
    rule_version_id → rule_versions.id NOT NULL,
    tag_id → tags.id NOT NULL,
    PRIMARY KEY (rule_version_id, tag_id)
)

rule_versions_race_details(
    version_id → rule_versions.id PRIMARY KEY,
    parent_race_id → rules.rule_id NULL,
    auto_calc_os BOOL DEFAULT false NOT NULL
)

```

```

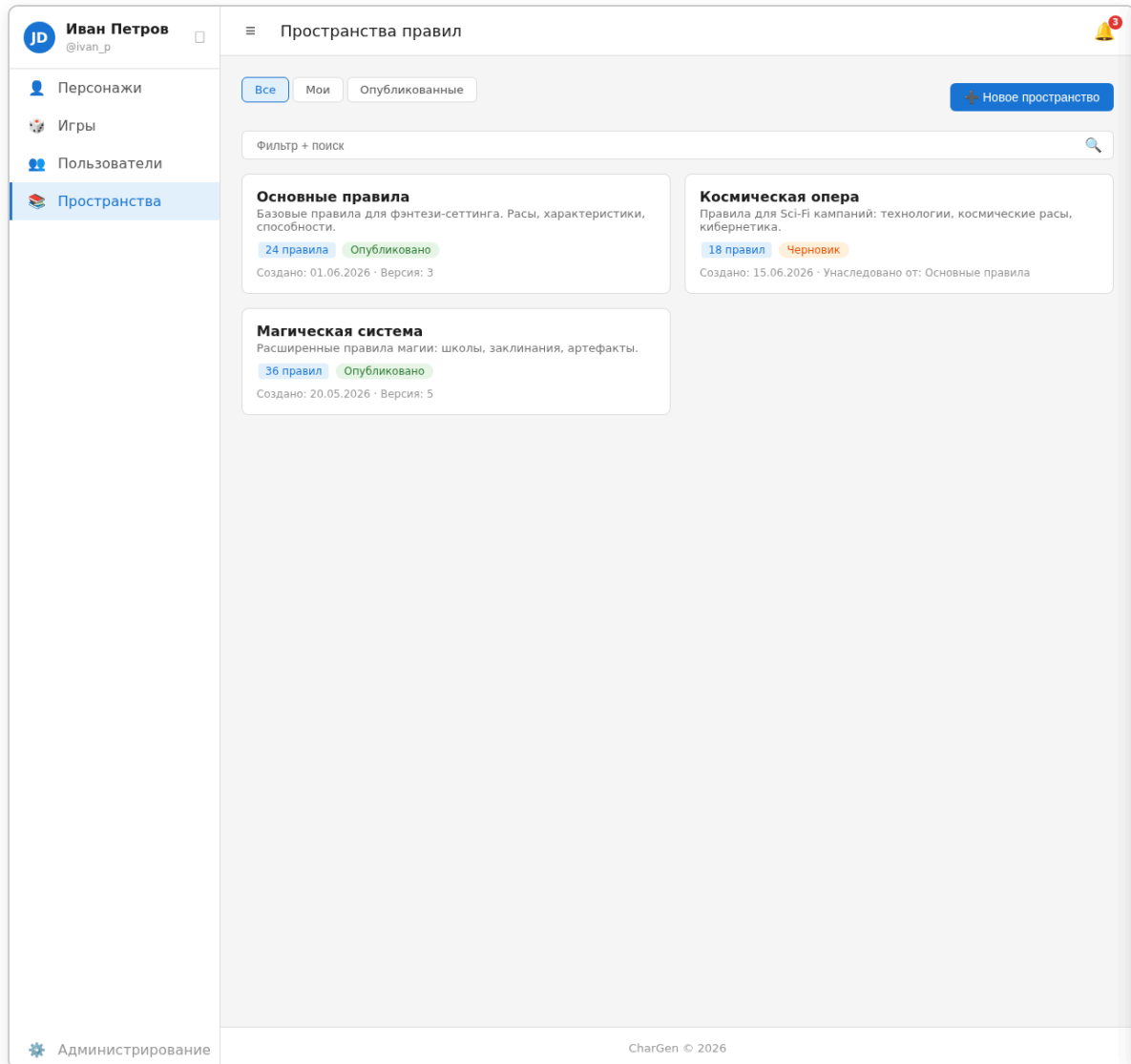
rule_versions_stat_details(
    version_id → rule_versions.id PRIMARY KEY,
    is_dimensional BOOL DEFAULT true NOT NULL,
    min_value INT, max_value INT,
    group_name VARCHAR,
    formula VARCHAR NULL
)

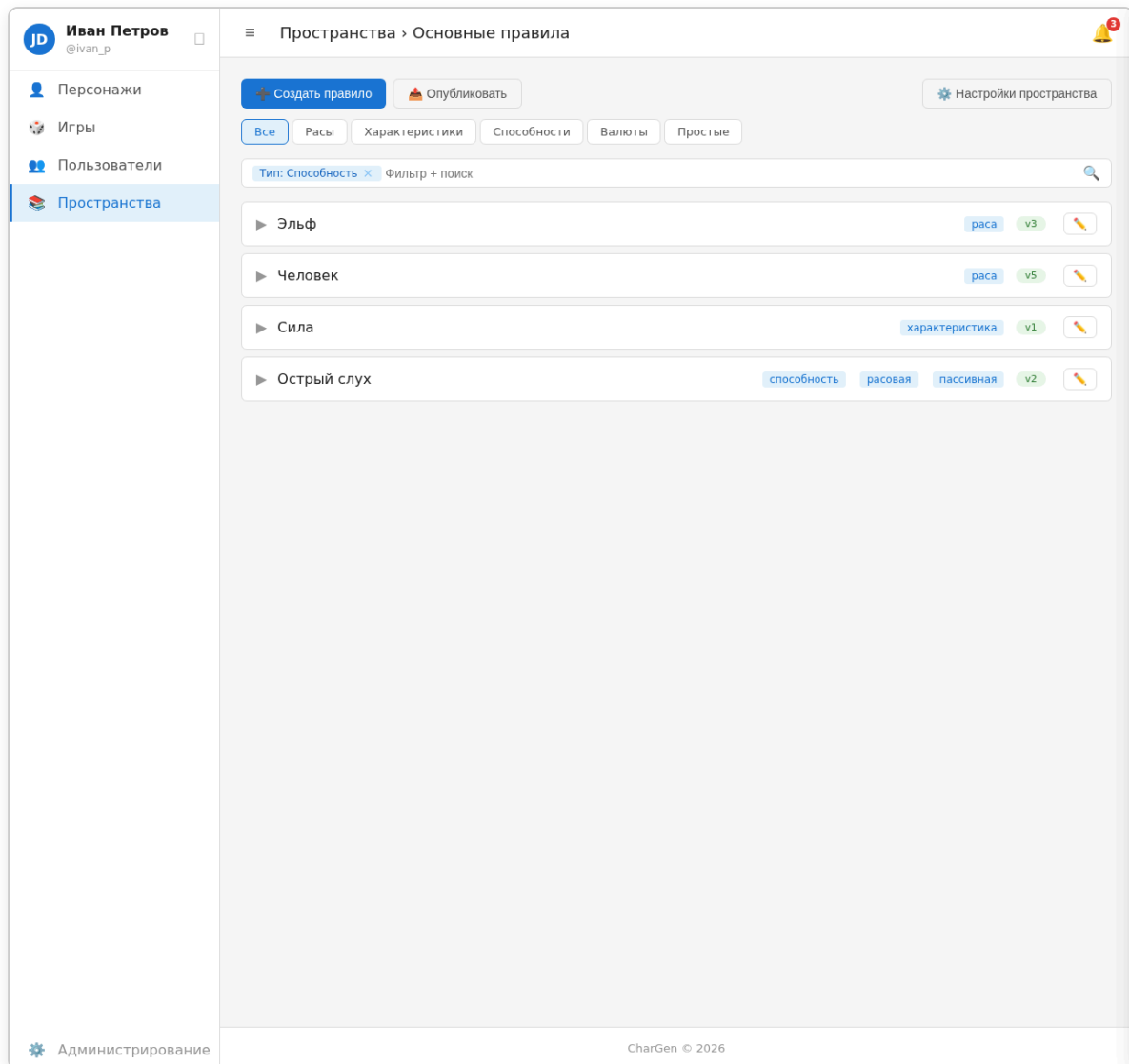
rule_versions_currency_details(
    version_id → rule_versions.id PRIMARY KEY,
    key VARCHAR UNIQUE NOT NULL,
    display_name VARCHAR NOT NULL
)

rule_versions_ability_details(
    version_id → rule_versions.id PRIMARY KEY,
    parent_ability_id → rules.rule_id NULL
)

rule_versions_item_details(
    version_id → rule_versions.id PRIMARY KEY,
    category VARCHAR NOT NULL,           -- money | equipment | other
    consumable BOOL DEFAULT false NOT NULL,
    weight_value INT, weight_size INT NULL,
    cost_gm INT NOT NULL,                -- стоимость в граммах меди
    durability INT NULL                  -- max прочность
)

```



Игры

```
games(
  id, name VARCHAR NOT NULL, description TEXT,
  short_description VARCHAR,                -- краткое описание для карточки в спи
  owner_id → users.id NOT NULL,
  space_id → spaces.id NOT NULL,
  rules_version_at TIMESTAMP NOT NULL,      -- срез версии правил
  status VARCHAR DEFAULT 'draft' NOT NULL, -- draft | open | closed_view | clos
  image_file_id → files.id NULL,
  os_points_limit INT NULL,
  or_points_limit INT NULL,
  money_limit INT NULL,                    -- бюджет на закупку предметов (в gm)
  tags_json JSON,                         -- теги игры (жанр, сеттинг, стиль)
  spec_json JSON,                         -- доп. ограничения (запретные теги и т.д.)
  active BOOL DEFAULT true NOT NULL,
  created_at
)
INDEX: (owner_id), (space_id), (status)
```

```

game_members(
  game_id → games.id NOT NULL,
  user_id → users.id NOT NULL,
  role VARCHAR DEFAULT 'player' NOT NULL, -- owner | gm | player
  PRIMARY KEY (game_id, user_id)
)
INDEX: (user_id) -- игры пользователя

game_member_permissions( -- инд. права игроков поверх роли
  game_id → games.id NOT NULL,
  user_id → users.id NOT NULL,
  permission_key VARCHAR NOT NULL, -- game.edit | game.moderate | game.
  PRIMARY KEY (game_id, user_id, permission_key)
)

game_invitations(
  id, game_id → games.id NOT NULL,
  inviter_id → users.id NOT NULL,
  invitee_id → users.id NOT NULL,
  status VARCHAR NOT NULL, -- sent | viewed | accepted | declin
  created_at
)
INDEX: (invitee_id, status), -- приглашения пользователя
      (game_id, status) -- приглашения игры

game_loot(
  id, game_id → games.id NOT NULL,
  item_rule_id → rules.rule_id NOT NULL,
  quantity INT NOT NULL,
  notes TEXT,
  status VARCHAR DEFAULT 'available' NOT NULL, -- available | acquired | distr
  created_at
)
INDEX: (game_id, status)

game_loot_interest(
  loot_id → game_loot.id NOT NULL,
  user_id → users.id NOT NULL,
  created_at,
  PRIMARY KEY (loot_id, user_id)
)

```

Персонажи

```

characters(
  id, name VARCHAR NOT NULL,
  owner_id → users.id NOT NULL,
  game_id → games.id NULL, -- NULL = свободный персонаж
  space_id → spaces.id NOT NULL,
  rules_version_at TIMESTAMP NOT NULL,
  status VARCHAR DEFAULT 'draft' NOT NULL, -- draft | ready | in_game | modera
  heir_of → characters.id NULL, -- группировка итераций одного перс
  active BOOL DEFAULT true NOT NULL,
  created_at
)
INDEX: (owner_id), (game_id), (status)

character_versions(

```

```

    id,
    character_id → characters.id NOT NULL,
    created_at TIMESTAMP NOT NULL,           -- версионный timestamp (аналог rule_versions)
    race_rule_id → rules.rule_id NOT NULL,   -- ID версии-оригинала при copy-on-write
    draft_of → character_versions.id NULL,    -- вся сборка персонажа (характеристики)
    data_json JSON NOT NULL,
    validated BOOL DEFAULT false NOT NULL
)
INDEX: (character_id, created_at DESC),      -- история версий персонажа
      (draft_of)                             -- поиск черновиков для сборки мусора

character_inventory(
    id,
    character_version_id → character_versions.id NOT NULL,
    item_rule_id → rules.rule_id NOT NULL,
    quantity INT NOT NULL,
    durability_left INT NULL,
    equipped BOOL DEFAULT false NOT NULL
)
INDEX: (character_version_id)

```

Уведомления

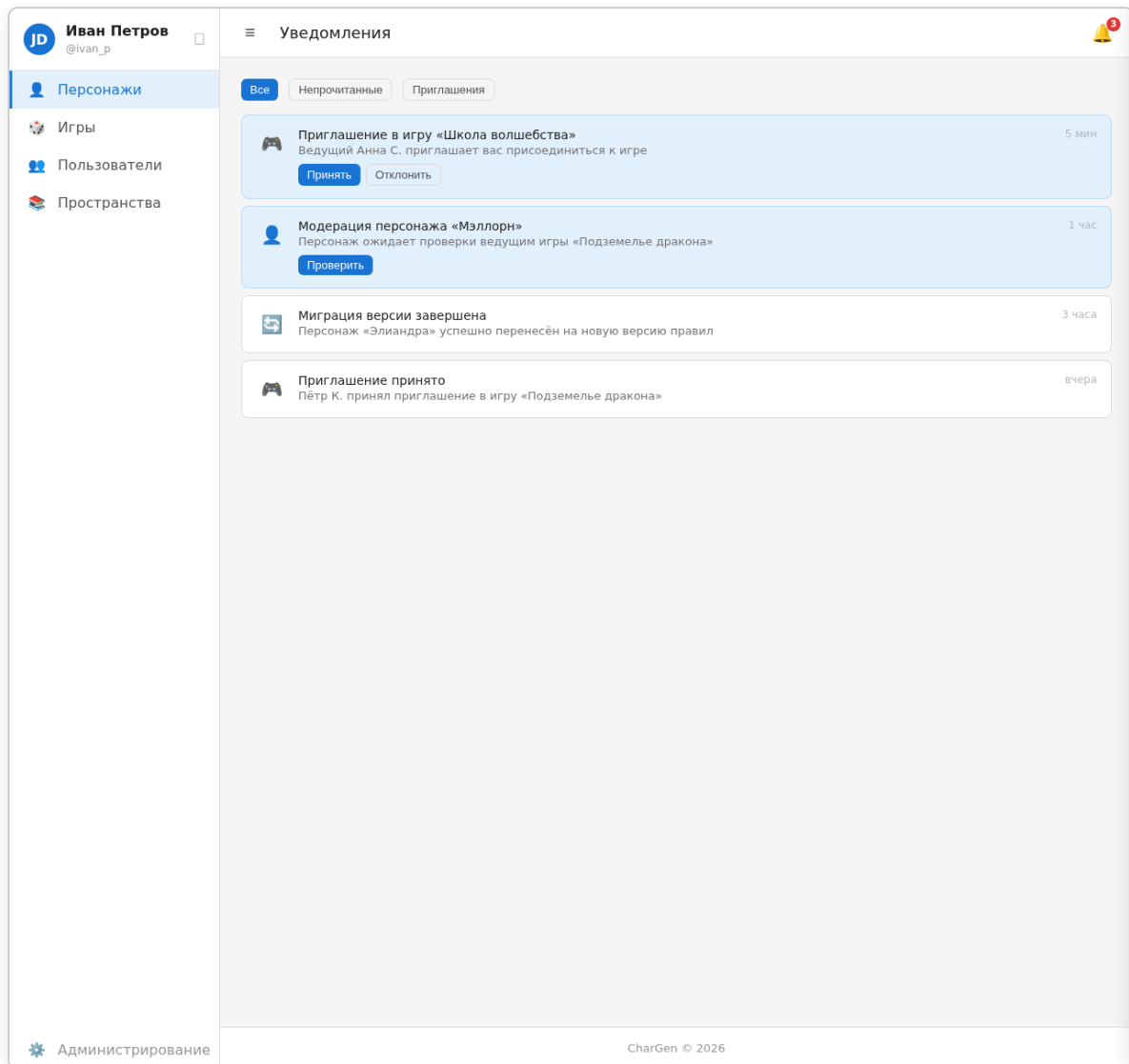
```

notification_templates(
    id, key VARCHAR UNIQUE NOT NULL,
    title_template VARCHAR NOT NULL,         -- плейсхолдеры: {{game_name}}
    body_template TEXT NOT NULL,             -- HTML с плейсхолдерами
    buttons_json JSON                        -- [{"label": "Принять", "action_type": "event", "url": "/game/accept"}]
)

notifications(
    id,
    from_user_id → users.id NULL,
    to_user_id → users.id NOT NULL,
    template_key → notification_templates.key NOT NULL,
    data_json JSON NOT NULL,                 -- {"game_name": "..."} для плейсхолдеров
    read BOOL DEFAULT false NOT NULL,
    read_at TIMESTAMP NULL,
    created_at
)
INDEX: (to_user_id, read, created_at DESC), -- лента уведомлений
      (to_user_id, read)                     -- счётчик непрочитанных

```

Макет: Уведомления (/notifications)



Чаты

```
chats(  
  id, type VARCHAR NOT NULL,           -- private | group | game  
  game_id → games.id NULL,  
  name VARCHAR NULL,  
  created_at  
)  
  
chat_members(  
  chat_id → chats.id NOT NULL,  
  user_id → users.id NOT NULL,  
  joined_at,  
  PRIMARY KEY (chat_id, user_id)  
)  
INDEX: (user_id)  
  
chat_messages(  
  id, chat_id → chats.id NOT NULL,  
  user_id → users.id NOT NULL,
```

```

    content TEXT NOT NULL,
    dice_result JSON NULL,          -- результат броска: {roll: ..., result
    created_at
)
INDEX: (chat_id, created_at DESC)

user_macros(
    id, user_id → users.id NOT NULL,
    name VARCHAR NOT NULL,
    text_template TEXT NOT NULL,
    roll_formula VARCHAR NOT NULL,  -- например "3d6"
    efficiency INT NOT NULL,
    created_at
)
INDEX: (user_id)

```

Схема утверждена. Индексы и типы полей могут уточняться на этапе реализации.

Что дальше (очередность реализации)

1. **Миграции БД** — создание таблиц по утверждённой схеме.
2. **Модели РНР** — Eloquent-модели (или аналог), репозитории для версионированных сущностей.
3. **Аутентификация** — страницы `/login`, `/register`, `/logout`, `/forgot-password`, `/reset-password/:token`. Сессии, middleware.
4. **CRUD пользователей и групп** — админ-интерфейс управления.
5. **CRUD пространств и правил** — редактор правил с версионированием, включая тип `item`.
6. **Раздел «Персонажи»** — создание, редактор (табы: раса → ОС → ОЛ → ОР → закупка), валидация.
7. **Раздел «Игры»** — управление играми, участниками, модерация, добыча.
8. **Уведомления** — шаблоны, отправка, лента.
9. **Чат** — чаты, команда `/roll`, макросы пользователей.
10. **Боевая карточка, журнал действий, diff версий** — после основных разделов.

*Очередность может уточняться.

Приложение А — Макеты интерфейса

Файлы макетов находятся в `makets/`. Страницы связаны навигацией (onclick-переходы).

Путь (ТЗ)	Файл макета	Статус
/login	makets/login.html	✓
/register	makets/register.html	✓
/forgot-password	makets/forgot-password.html	✓
/games	makets/games-list.html	✓
/games/new	makets/game-new.html	●
/games/:id	makets/game-detail.html	✓
/games/:id/characters	makets/game-detail-characters.html	✓
/games/:id/moderate	makets/game-moderate.html	●
/characters	makets/characters-list.html	✓
/characters/:id	makets/character-card.html	●
/characters/new	makets/character-new.html	●
/characters/:id/edit	makets/character-editor.html	✓
/characters/:id/edit?step=race	makets/character-editor-race.html	✓
/spaces	makets/spaces-list.html	✓
/spaces/new	makets/space-new.html	●
/spaces/:id	makets/space-rules.html	✓
/spaces/:id/settings	makets/space-settings.html	●
/spaces/:id/publish	makets/space-publish.html	●
/spaces/:id/rules/:ruleId	makets/space-rule-view.html	●
/notifications	makets/notifications.html	✓
/users/:id/edit	makets/profile-edit.html	✓
/characters/:id/edit?step=inventory	makets/character-inventory.html	✗
/games/:id/loot	makets/game-loot.html	✗
/chat/:id	makets/chat.html	✗
/profile/macros	makets/profile-macros.html	✗

Путь (ТЗ)	Файл макета	Статус
Остальные страницы ТЗ	—	 не реализован